University of Pavia, AA 2025/2026

Computational methods in Physics

Simone Rodini simone.rodini@unipv.it

Abstract

Lecture notes for the Computational methods in Physics class, within the Physical Sciences master degree of University of Pavia.

These notes are brief, and their purpose is to collect the main results and expressions, so that they can be used as a reference for an actual implementation.

With the notes are available a number of Mathematica notebooks, that serves as support material.

Contents

I Approximation and differentiation 3

1 Polynomial approximation 3

2 Nodal and Modal expansions 4
- 2.1 Nodal expansion . . . 4
- 2.2 Modal expansion . . . 4

3 Barycentric forms 6

4 Runge phenomenon 7

5 Chebyshev polynomials 7

6 Chebyshev barycentric weights 10

7 Differentiation on the grid 11

8 Non-linear mappings 14
- 8.1 First example . . . 14
- 8.2 Second example . . . 16
- 8.3 Final results . . . 16

9 Beyond single-grid construction 16
- 9.1 C^0 tessellation . . . 17
- 9.2 Sliding window . . . 20
  - 9.2.1 Further generalization . . . 21

10 Integration on the grid 22

II Numerical Quadrature 24

11 Gauss formulas 25

12 Rule extensions and error estimate 27

13 Clenshaw-Curtis rules 28

14 Ogata quadrature 29

15 On the Cauchy principal Value 31

16 On (semi-)infinte intervals of integration 32

17 Monte Carlo 33
- 17.1 Crude Monte Carlo . . . 33
- 17.2 Hit-or-miss Monte Carlo . . . 34
- 17.3 Importance sampling . . . 35

III Integration of Differential Equations 36

Bibliography 37

Part I

Approximation and differentiation

1 Polynomial approximation

In the course, we will only cover the case of mono-dimensional (or univariate) problems. Let us simply mention that the multivariate case, with factorized domains (i.e. squares, cubes, etc) can be trivially obtained by combining multiple univariate constructions. If the domain is non-factorizable, then dedicated methods should be employed.

Suppose that we want to obtain a reliable approximation of a function $f(x)$, which might be very expensive to compute, so that we can generate the approximation once, and then use it instead of the function itself. This will be particularly useful for solving quadrature problems, and certain types of differential equations.

The point is how to construct such an approximation. We will take the general approach of using polynomials to approximate our solution. How to construct a polynomial in practice, we will see in a bit. For the moment, we state the Weierstrass approximation theorem, upon which relies the fact that we can expect a polynomial to perform sufficiently well as an approximant:

Weierstrass approximation theorem: If f is any continuous function on the finite closed interval $[a, b]$, then for every $\varepsilon > 0$ there exists a polynomial $p_n(x)$ of degree $n = n(\varepsilon)$ such that

$$\max_{x\in[a,b]} |f(x) - p_n(x)| < \varepsilon \tag{1}$$

□

There are two, conceptually distinct ways to go about finding a polynomial approximation of a given function. Both approaches rely on the definition of a polynomial basis that will define how the approximant is constructed. We will cover number of different basis, but the most famous are

$$\mathcal{M}_p = \{1, x, x^2, ..., x^p\} \tag{2}$$

which is nothing more than the natural basis for the polynomials of degree $\leq p$;

$$\mathcal{L}_p\left(\{x_i\}_{i=0}^p\right) = \{l_j(x; \{x_i\}_{i=0}^p)\}_{j=0}^p, \qquad l_j(x; \{x_i\}_{i=0}^p) = \prod_{\substack{i=0\\ i\neq j}}^{p} \frac{x - x_i}{x_j - x_k} \tag{3}$$

that is the Lagrange basis for the given set of nodes $\{x_i\}_{i=0}^p$, where note that the nodes are exactly $p+1$, where p is the degree of each of the polynomials $l_j(x)$;

$$\{P_n(x)\}_{n=0}^p \tag{4}$$

which are the Legendre polynomials of degree $\leq p$. Note that the Legendre polynomials are just one example of a special class of polynomial families that are orthogonal with respect to some given inner product. In this case, the inner product looks like

$$\langle P_n | P_m \rangle = \int_{-1}^{1} dx P_n(x) P_m(x) = \frac{2}{2n+1}\delta_{n,m} \tag{5}$$

Let us note that neither the natural basis not the Lagrange basis are, in general, orthogonal basis.

Now, we can define the two different methods to find a polynomial approximation of a function $f(x)$ as nodal expansions versus modal expansions. The former is the easier to understand.

2 Nodal and Modal expansions

2.1 Nodal expansion

Fix a polynomial basis of degree p, with basis elements $b_j(x)$ then we can construct an approximation

$$\phi(x) = \sum_{j=0}^{p} c_j b_j(x) \quad \text{such that} \quad f(x_i) = \phi(x_i) \tag{6}$$

for a given set of exactly $p+1$ distinct nodes x_i. Looking at the requirements, this should immediately point towards the Lagrange polynomials, and indeed they are essentially the polynomial basis of choice for nodal expansions. It should also be apparent why the expansion is named nodal: we are fixing a set of nodes, on which we require that the approximation and the function itself are exactly equal. We can construct a bit of intuition why the Lagrange basis is the most obvious choice in this case. First of all, by the definition itself we see that

$$l_j(x_i) = \delta_{ij} \tag{7}$$

then, we have

$$\phi(x) = \sum_{j=0}^{p} c_j l_j(x) \Rightarrow \phi(x_i) = \sum_{j=0}^{p} c_j l_j(x_i) = \sum_{j=0}^{p} c_j \delta_{ij} = c_i \tag{8}$$

therefore, using the Lagrange basis, we immediately have that $c_i = f(x_i)$, which gives us a very straightforward way to construct the polynomial interpolation.

What would have happened if we were to choose a different basis? For instance, let us take the canonical polynomial basis:

$$\phi(x) = \sum_{j=0}^{p} c_j x^j \Rightarrow \phi(x_i) = \sum_{j=0}^{p} c_j x_i^j \tag{9}$$

Now, we can use a very common 'trick': let us collect $\phi(x_i)$ into a vector: $\boldsymbol{\phi} = \{\phi(x_0), ..., \phi(x_p)\}$, and let us do the same for the coefficients $\boldsymbol{c}$ and for the matrix $B_{ij} = x_i^j$. Then, the system of equations to determine the coefficients becomes

$$\boldsymbol{\phi} = B\boldsymbol{c} \Rightarrow \boldsymbol{c} = B^{-1}\boldsymbol{\phi} \tag{10}$$

Since we choose a basis for our polynomials, and we have the constant assumption that all the nodes are different from each other, then the matrix is guaranteed to be invertible (a zero eigenvalue in the matrix would signify a non-trivial set of nodes for which the linear independence of the basis functions would be violated, hence a contradiction). The problem is that, numerically, taking the inverse is both slow and potentially very inaccurate.

This procedure is not limited to the two set of polynomial basis, but can actually be employed for any polynomial basis, with the only modification being in the matrix B. We can also immediately see that the Lagrange basis is nothing more than the basis that diagonalizes the matrix B directly.

2.2 Modal expansion

The nodal expansion works in a rather general context, since the only required input is the choice of the polynomial basis and the selection of the nodes. The modal expansion instead requires the specification of the polynomial basis and an inner product on the space of polynomials. We saw that the Legendre polynomials are orthogonal with respect to the inner product with the unit function on the interval $[-1, 1]$.

As for the case of the nodal expansion, where the Lagrange polynomials played a special role, so here the entire class of orthogonal polynomials play a special role. Let us write again the polynomial approximation as

$$\phi(x) = \sum_{j=0}^{p} c_j b_j(x) \tag{11}$$

but now we want to derive the expansion coefficients by imposing the equality between the approximant and exact moments (or modes) of the function, i.e.

$$\langle f|b_j\rangle_\omega = \langle \phi|b_j\rangle_\omega \tag{12}$$

where ω defines the inner product as

$$\langle f|g\rangle_\omega = \int_{-1}^{1} dx\omega(x)f(x)g(x) \tag{13}$$

where we assume that the function $\omega(x) \geq 0$ for all $x \in [-1, 1]$. Note that here we will always assume that the domain is $[-1, 1]$. One can move to arbitrary compact domain via a simple linear transformation.

Here it becomes rather evident why orthogonal polynomials are the convenient choice: from

$$\langle b_i|b_j\rangle_\omega = n_i\delta_{i,j} \tag{14}$$

where n_i is some normalization, we immediately have that

$$\langle \phi|b_j\rangle_\omega = n_j c_j \Rightarrow c_j = \frac{1}{n_j}\langle f|b_j\rangle_\omega \tag{15}$$

We note that, typically, it is more computationally heavy determine the coefficient of a modal expansion compared to a nodal one. However, in some context it is very convenient being able to determine the expansion of an arbitrary function onto a basis of polynomials. As an example, consider the following integro-differential problem that emerges in the study of perturbative-QCD

$$\frac{\partial f(x,\mu)}{\partial \log \mu^2} = \int_x^1 dz C(z) f\left(\frac{x}{z}\right) \tag{16}$$

where $f(x) \neq 0$ in $[0, 1]$. Note that this problem cannot really be solved in this form. Then, assume that we are happy with an approximated solution, specifically, we are happy to have a polynomial expansion in the canonical basis. We could try to construct a nodal approximation (which is one of the current methods employed to solve such equations to a high degree of accuracy) or we could try to use a modal expansion. With the latter, we have

$$\begin{aligned}\frac{\partial \langle f(x,\mu)|x^n\rangle}{\partial \log \mu^2} &= \int_0^1 dx x^n \int_x^1 dz C(z) f\left(\frac{x}{z}\right) \\ &= \int_0^1 dz C(z) \int_0^z dx x^n f\left(\frac{x}{z}\right) \\ &= \int_0^1 dz C(z) z^{n+1} \int_0^1 dt t^n f(t)\end{aligned} \tag{17}$$

or, in other words

$$\frac{\partial \gamma_n(\mu)}{\partial \log \mu^2} = C_n \gamma_n(\mu) \tag{18}$$

where $\gamma_n(\mu) \equiv \langle f(x,\mu)|x^n\rangle$ and

$$C_n = \int_0^1 dz C(z) z^{n+1} \tag{19}$$

Now this is trivially solved as

$$\gamma_n(\mu) = \left(\frac{\mu}{\mu_0}\right)^{2C_n} \gamma_n(\mu_0) \tag{20}$$

This is especially powerful since, in practice, one may only be able to compute C_n instead of the full function $C(z)$, because the extra integral, simplifies the computation of the corresponding perturbative diagrams.

3 Barycentric forms

The naïve construction of the Lagrange interpolating polynomial

$$l_j(x) = \prod_{\substack{i=0 \\ i \neq j}}^{p} \frac{x - x_i}{x_j - x_i} \tag{21}$$

is actually not very good in practice, both from a numerical stability point of view and from a performance point of view.

The first improvement that we can make to the formula is to note that we can precompute, once the grid is known, the weights

$$\lambda_j = \prod_{\substack{i=0 \\ i \neq j}}^{p} (x_j - x_i)^{-1} \tag{22}$$

Moreover, we can define a unique polynomial of degree $p+1$ that has zeros on exactly the $p+1$ nodes of the grid

$$l(x) = \prod_{j=0}^{p} (x - x_j) \tag{23}$$

which is sometimes referred to as the node polynomial. Now, we can combine this two constructions to obtain a fairly simple alternative representation for the Lagrange polynomials

$$l_j(x) = \frac{\lambda_j l(x)}{x - x_j} \tag{24}$$

this is called the <u>first barycentric formula</u> of the Lagrange interpolant. What is the computational advantage here?

In the original formulation, to compute the approximant of a function

$$p(x) = \sum_{j=0}^{p} f_j l_j(x) \tag{25}$$

we needed to take $O(p)$ additions, and for each addition we needed $O(p)$ multiplications, therefore the total computational complexity comes out to be $O(p^2)$, which is typically not very good. With the first barycentric formula instead, we still need $O(p^2)$ operations to compute the weights λ_j, but this is a one-time-only computation, therefore it does not really matter that much. Once we have them, we do not have anymore nested sums and products: the product that defines $l(x)$ factorizes from the sum over j:

$$p(x) = l(x) \sum_{j=0}^{p} \frac{\lambda_j f_j}{x - x_j} \tag{26}$$

therefore the total computational complexity comes out to be $O(p + p) \sim O(p)$.

A useful observation is that we can obtain the weights as the derivative of the node polynomial evaluated on the grid nodes. Specifically

$$\lambda_j = \frac{1}{l'(x_j)} \tag{27}$$

The proof is by direct computation, and is left as a simple exercise.

There exist an even simpler formula though. We can notice that the polynomial approximant for the function $g(x) = 1$ is

$$\sum_{j=0}^{p} l_j(x) \tag{28}$$

but since the function itself is a polynomial, and we know that the polynomial interpolant is unique, we have immediately that

$$\sum_{j=0}^{p} l_j(x) = 1 \Rightarrow l(x) \sum_{j=0}^{p} \frac{\lambda_j}{x - x_j} = 1 \Rightarrow l(x) = \left[\sum_{j=0}^{p} \frac{\lambda_j}{x - x_j} \right]^{-1} \tag{29}$$

hence

$$p(x) = \left[\sum_{j=0}^{p} \frac{f_j \lambda_j}{x - x_j} \right] \left[\sum_{j=0}^{p} \frac{\lambda_j}{x - x_j} \right]^{-1} \tag{30}$$

which is called the second barycentric formula of the Lagrange interpolant.

There are proofs about the stability of the two formulas for different choices of the underlying grid. Specifically, for an arbitrary grid the first barycentric formula has stronger stability properties compared to both the naïve formulation and the second barycentric formula. It also behave quite well under extrapolation outside the interpolation domain. However, it suffers from potentially catastrophic finite-precision problems (under- and over-flows due to the finite precision of the representation of numbers on a computer). If one steers away from generic grids and restrict itself to special types of grid that have their points clustered near the edges of the domain (more on in in a moment), then the second barycentric formula works very well inside the interpolation region without suffering from finite-precision problems, whereas it is still not stable for the extrapolation region.

Conceptually, the main advantage of the second barycentric formula is that one can uniformly scale all the weights by some constant amount $\lambda_j \to c\lambda_j \ \forall j$ and the expression is unaffected. This is not true for the first barycentric formula, which has a linear dependence over the weights. It is simple to observe that the weights grow extremely fast (exponentially fast in fact) with the number of nodes in the grid. Therefore, once we have a lot of nodes, the weights are so big that cannot really be represented on a computer.

4 Runge phenomenon

To convince ourselves that using equispaced grids is almost never a good idea, we can try to interpolate the function

$$\frac{1}{1 + 25x^2} \tag{31}$$

on the standard interval $[-1, 1]$. If one does it on a grid of equispaced points, the more points one uses the worse the approximation of the function near the boundary of the intervals!

5 Chebyshev polynomials

One of the central basis of orthogonal polynomials in approximation theory is the basis defined by the Chebyshev polynomials. These can be defined in different ways. The most compact representation, which seems a bit obscure at first glance, is

$$T_n(\cos(\theta)) = \cos(n\theta) \tag{32}$$

where we note that the support of the Chebyshev polynomials is always assumed to be the standard interval $[-1, 1]$. With this representation, it can be verified that they are orthogonal with respect to the scalar product

$$\begin{aligned} \langle T_n | T_m \rangle_{1/\sqrt{1-x^2}} &= \int_{-1}^{1} \frac{dx}{\sqrt{1-x^2}} T_n(x) T_m(x) \\ &= \int_0^{\pi} d\theta \cos(n\theta) \cos(m\theta) = \frac{\pi}{2} \delta_{nm} \left(1 + \delta_{n0}\right) \end{aligned} \tag{33}$$

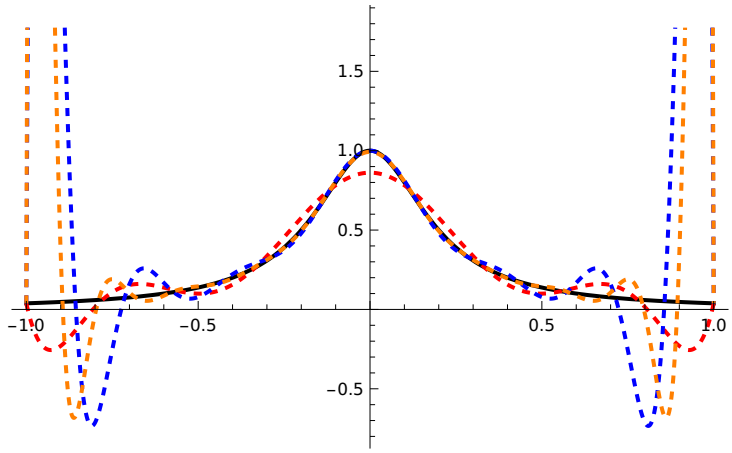

Figure 1: Exemplification of the Runge phenomenon. Solid black line is the true function, then in red, blue and orange respectively the interpolations with 10, 15, and 20 nodes.

Although nice and compact, this representations is anything but clear. How can we be sure that T_n is in fact a polynomial? For once, we can immediately see that

$$T_0(\cos(\theta)) = 1, \quad T_1(\cos(\theta)) = \cos(\theta) \tag{34}$$

i.e.

$$T_0(x) = 1, \quad T_1(x) = x \tag{35}$$

Now, we can state the an inductive construction for the Chebyshev polynomials as

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \tag{36}$$

This can be easily verified to hold true for the cos representation above. And this recurrence relation clearly shows that T_n is a polynomial in its inputs for all n.

The more or less natural question is now: why the cosine? To understand it, assume to have a function f defined from our standard interval to the real numbers that is regular enough. Then, we can always write f as a modal expansion onto the basis of Chebyshev polynomials as

$$f(x) = \sum_{j\geq 0} a_j T_j(x) \tag{37}$$

Now consider a related function, that is defined by $F(\theta) = f(\cos(\theta))$, for which we obviously have that $F(\theta) = F(-\theta)$. For this function we can write the Fourier series as

$$F(\theta) = \sum_{k\in\mathbb{Z}} c_k e^{ik\theta} \tag{38}$$

where we have a series and not an integral because we are working on a compact interval and therefore have only numerably many distinct momentum modes, instead of a continuum spectrum. Since we know that the function is even, we reduce the Fourier series in half, by noticing that the negative momentum modes must be identical to the positive ones, i.e.

$$F(\theta) = \sum_{k\geq 0} c_k \left(e^{ik\theta} + e^{-ik\theta}\right) = \sum_{k\geq 0} 2c_k \cos(k\theta) = \sum_{k\geq 0} 2c_k T_k(\cos(\theta)) \tag{39}$$

But we also have that

$$F(\theta) = f(\cos(\theta)) = \sum_{j\geq 0} a_j T_j(\cos(\theta)) \tag{40}$$

Since the expansion is done on a polynomial basis we automatically have that

$$2c_k = a_k \quad \forall k \tag{41}$$

In essence, the expansion on a Chebyshev basis is nothing more that a discrete cosine transform!

To make the connection complete, we need to specify what we mean by f being 'regular enough'. It is to consider f to be of uniform bound variation on the interval, which is to say that it should be Lipschitz-1, i.e. $\exists C$ such that $|f(x) - f(y)| \leq C|x - y|$ $\forall x, y \in [-1, 1]$. In words, we can always find a finite constant C so that the variation of f on the interval is bounded by the separation between the points times the constant.

Then we can state the theorem
Chebyshev Series if $f : [-1, 1] \to \mathbb{R}$ is Lipschitz-1, then it has a unique Chebyshev series, which converges both absolutely and uniformly. $\square$

Leveraging again the connection between the Fourier representation and the Chebyshev one, we can introduce the Chebyshev-Lobatto points. To do so, fix an integer n, and consider the $n + 1$ equispaced points on the top-half of the unit circle, so that $\theta_j = \pi(j/n)$ for $j = 0, .., n$. The cartesian coordinates of these points are clearly $(\cos(\theta_j), \sin(\theta_j))$. If we take the angles θ_j to be grid nodes for the Fourier representation, then we immediately have that grid nodes for the Chebyshev representation are the cosines of those angles, i.e.

$$t_j = \cos(\theta_j) = \cos\left(\frac{j\pi}{n}\right) \tag{42}$$

It is immediate to see that these are the unique $n + 1$ points on which $T_n(x) = \pm 1$.

Why are we introducing these points? To answer we need to consider two different ways to construct an approximation of a function f using Chebyshev polynomials.

On the one side, we could use directly the modal expansion, truncated to some maximum degree, which is nothing more that the partial sum of the Chebyshev series. Let us define it as

$$f_p(x) = \sum_{j=0}^{p} a_j T_j(x) \tag{43}$$

On the other side, we want to construct a nodal interpolation, on the Chebyshev-Lobatto points. We know that the Lagrange polynomial is the unique interpolant. We also know that the Lagrange polynomials form a basis for the polynomials of a given degree, and so does the Chebyshev basis. Therefore, we can write the interpolant using the Chebyshev polynomials instead of the Lagrange ones as

$$p_n(x) = \sum_{j=0}^{p} c_j T_j(x) \tag{44}$$

Do not be tempted to interpret immediately $c_j = a_j$! c_j are defined by the interpolating construction, i.e.

$$p_n(t_k) = f(t_k) \tag{45}$$

which will give a system of linear equations in the c_j, whereas there is no guarantee that $f_p(t_k) = f(t_k)$!

However, intuition says that some connection must exist. Indeed, we can prove by direct computations that for fixed $p \geq 1$ and for any $m \in [0, p]$ the Chebyshev polynomials $T_{2jp\pm m}$ take the same values as T_m on the $p + 1$ Chebyshev-Lobatto points for all $j > 0$.

With this, we can take the full Chebyshev of f, evaluate it in turn on each Chebyshev-Lobatto point as use the statement above, together with the nice convergence properties of the series, to reorder terms. Specifically, in the full series we re-arrange terms so that we have a tower of combinations of Chebyshev polynomials evaluated on the grid points on which they all take the same values. This defines the coefficients c_k of the interpolant in terms of the coefficients of the full series as

$$c_0 = a_0 + \sum_{j\geq 1} a_{2jp} \tag{46}$$

$$c_k = a_k + \sum_{j\geq 1} a_{k+2jp} + a_{-k+2jp} \tag{47}$$

$$c_p = a_p + \sum_{j\geq 1} a_{p+2jp} \tag{48}$$

This, by construction, ensures that

$$p_n(x) = \sum_{j=0}^{p} c_j T_j(x) \tag{49}$$

interpolates the function on the grid.

This is obviously not a practical recipe to compute the interpolant! It is, however, very important from the theory point of view, because it ensures that the interpolants converge absolutely to the original function, which is a very useful property to have.

6 Chebyshev barycentric weights

Another major advantage of the Chebyshev-Lobatto points, is they allow for a closed-form computation of the interpolation weights, which immensely reduces the number of computations required in practice, not to mention the numerical stability of the procedure.

Let us start with some intermediate results. First, we have a representation for the Lagrange polynomial

$$l(x) = 2^{-p}\left(T_{p+1}(x) - T_{p-1}(x)\right) \tag{50}$$

To prove it, we know that $l(x)$ is defined by its $p+1$ roots, that for us are the $p+1$ Chebyshev-Lobatto points t_j. Since the polynomial on the right-hand side is also of degree $p+1$, if we can show that has the exact same roots, we have established the proportionality. Then, since the coefficient of x^{p+1} of $l(x)$ is exactly 1, we can just show that the same is true for the normalized right-hand side, and we are done.

First, let us compute

$$T_{p+1}(t_j) = \cos\left(\frac{(p+1)j\pi}{p}\right) = \cos(j\pi)\cos\left(\frac{j\pi}{p}\right) - \sin(j\pi)\sin\left(\frac{j\pi}{p}\right) \tag{51}$$

$$T_{p-1}(t_j) = \cos\left(\frac{(p-1)j\pi}{p}\right) = \cos(j\pi)\cos\left(\frac{j\pi}{p}\right) + \sin(j\pi)\sin\left(\frac{j\pi}{p}\right) \tag{52}$$

$$\tag{53}$$

Since $\sin(j\pi) = 0$, we immediately have that $T_{p+1}(t_j) - T_{p-1}(t_j) = 0$ for all $j \in [0, p]$. Now, from the defining recurrence relation of the Chebyshev polynomials we have that

$$T_n(x) = 2^{n-1}x^n + ... \tag{54}$$

if $n > 0$, from which immediately follows that

$$T_{p+1}(x) - T_{p-1}(x) = 2^p x^{p+1} + ... \tag{55}$$

from which the equality is established.

Now we can compute the weights. Recall that they can be defined from the derivatives of the Lagrange polynomial at the grid nodes, i.e.

$$\beta_j = \frac{1}{l'(t_j)} = \frac{2^p}{T'_{p+1}(t_j) - T'_{p-1}(t_j)} \tag{56}$$

where we switched to a dedicated notation for the weights, to remember that these are for the Chebyshev points only.

Now, we can compute the derivatives of the Chebyshev polynomials, using a very simple trick:

$$\frac{d}{dx}T_n(x) = \frac{d}{d\cos\theta}T_n(\cos\theta) = -\frac{1}{\sin\theta}\frac{d}{d\theta}T_n(\cos\theta) \Rightarrow \tag{57}$$

$$
\begin{aligned}
T'_{p\pm1}(t_j) &= -\frac{1}{\sin\theta_j}\frac{d}{d\theta_j}\cos\left((p\pm1)\theta_j\right) \\
&= -\frac{1}{\sin\theta_j}\frac{n}{\pi}\frac{d}{dj}\cos\left(j\pi\pm\frac{j\pi}{p}\right) \\
&= \frac{1}{\sin\theta_j}\frac{n}{\pi}\left(\pi\pm\frac{\pi}{p}\right)\left[\sin(j\pi)\cos\left(\frac{j\pi}{p}\right)\pm\cos(j\pi)\sin\left(\frac{j\pi}{p}\right)\right]
\end{aligned}
\tag{58}
$$

Now, if $j \neq 0, p$ we can trivially evaluate the expression, and get

$$
T'_{p\pm1}(t_j) = \pm(-1)^j(p\pm1) \tag{59}
$$

if $j = 0, p$, we need to be a bit more careful because the first one over sine diverges. No matter, taking the appropriate limit (by symmetry we have that the results for $j = p$ must be equal, up to a global $(-1)^p$ to the results of $j = 0$):

$$
T'_{p\pm1}(t_0) = (p\pm1)^2 \tag{60}
$$

Therefore, we have that

$$
\beta_0 = \frac{2^{p-1}}{p}\frac{1}{2} \tag{61}
$$

$$
\beta_{j\neq0,p} = \frac{2^{p-1}}{p}(-1)^j \tag{62}
$$

$$
\beta_p = \frac{2^{p-1}}{p}\frac{(-1)^p}{2} \tag{63}
$$

It is immediate to see that we can define a scaled down version of the weights by

$$
\tilde{\beta}_j = \frac{p}{2^{p-1}}\beta_j \tag{64}
$$

It is also convenient for the following discussions to extract the phase from the weights, and write

$$
\gamma_j = \tilde{\beta}_j(-1)^j \tag{65}
$$

having essentially that $\gamma_{0,p} = 1/2$ and $\gamma_{j\neq0,p} = 1$.

Therefore, our basis functions for the Chebyshev interpolant on the Chebyshev-Lobatto points is greatly simplified, especially in the second barycentric representation:

$$
p(x) = \sum_{j=0}^{p} f(t_j)b_j(x), \quad b_j(x) = \frac{\tilde{\beta}_j}{x-t_j}\Bigg/\sum_{i=0}^{p}\frac{\tilde{\beta}_j}{x-t_j} \tag{66}
$$

Now we can establish a connection between the coefficients $f(t_j)$ of the interpolant written in the natural Lagrange basis, and the coefficients c_k for the interpolant written in the Chebyshev basis. This will not be immediately useful, but we will need it when we will talk about quadrature rules. We can write

$$
p(x) = \sum_{j=0}^{p} f(t_j)b_j(x) = \sum_{j=0}^{p} c_j T_j(x) \tag{67}
$$

with

$$
c_j = \frac{2\gamma_j}{p}\sum_{i=0}^{p}\gamma_i T_j(t_i) f_i \tag{68}
$$

7 Differentiation on the grid

Now that we discussed the different ways to construct a polynomial interpolant, we can apply this construction to obtain a method to compute the derivatives of our function in an approximated way.

First, let us derive the results for the generic Lagrange interpolant, and then we will derive specialized representations for the Chebyshev grids.

First the trivial expression

$$p(x) = \sum_{j=0}^{p} f_j l_j(x) \Rightarrow p'(x) = \sum_{j=0}^{p} f_j l_j'(x) \tag{69}$$

but then $p'(x)$, as a function, can be itself interpolated on the grid, to give the representation

$$p'(x) = \sum_{j=0}^{p} p'(x_j) l_j(x) \tag{70}$$

We can use now the first representation into the second, to obtain

$$p'(x) = \sum_{j=0}^{p} l_j(x) \sum_{i=0}^{p} f_i l_i'(x_j) \tag{71}$$

Define the derivative matrix

$$D_{ij} = l_i'(x_j) \tag{72}$$

and we can write

$$f'(x) \sim p'(x) = \sum_{i=0}^{p} f_i \sum_{j=0}^{p} D_{ij} l_j(x) \tag{73}$$

i.e. we have identified

$$l_i'(x) = \sum_{j=0}^{p} D_{ij} l_j(x) \tag{74}$$

which also gives us a formula for repeated differentiation: with the convention that repeated indexes are summed over

$$\frac{d^n}{dx^n} l_{i_0}(x) = D_{i_0 i_1} D_{i_1 i_2} .. D_{i_{n-1} i_n} l_{i_n}(x) \tag{75}$$

Now we can explicitly compute the derivative matrix. First, we note that we have

$$\begin{aligned} l_j'(x) &= \frac{d}{dx} \prod_{\substack{k=0 \\ k \neq j}}^{p} \frac{x - x_k}{x_j - x_k} \\ &= \sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x_j - x_n} \prod_{\substack{k=0 \\ k \neq j,n}}^{p} \frac{x - x_k}{x_j - x_k} \\ &= \left[\sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x - x_n} \right] l_j(x) \end{aligned} \tag{76}$$

Now we can use this expression to evaluate it at the grid points. First, we have the diagonal terms

$$l_j'(x_j) = \left[\sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x_j - x_n} \right] l_j(x_j) = \sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x_j - x_n} \tag{77}$$

For the off-diagonal terms we instead obtain

$$l_j'(x_i) = \left[\sum_{\substack{n=0 \\ n \neq j}}^{p} \frac{1}{x_j - x_n} \right] \prod_{\substack{k=0 \\ k \neq j,n}}^{p} \frac{x_i - x_k}{x_j - x_k} = \frac{1}{x_j - x_i} \prod_{\substack{k=0 \\ k \neq j,i}}^{p} \frac{x_i - x_k}{x_j - x_k} \tag{78}$$

where the last equality holds because if $\forall n \neq i$ in the sum, the product contains a single term in the numerator that is $x_i - x_i$, hence it removes the corresponding contribution.

Notice that we obtained the results in the naïve representation for the Lagrange interpolant. It is actually easy to see that an equivalent presentation of the final result is

$$f'(x) \sim p'(x) = \sum_{j=0}^{p} l_j(x) \sum_{i=0}^{p} f_i D_{ij} \tag{79}$$

with this we can define the coefficients $\tilde{f}_j = \sum_{i=0}^{p} f_i D_{ij}$, and with these apply the first or second barycentric formulas. For instance

$$p'(x) = l(x) \sum_{j=0}^{p} \frac{\tilde{f}_j \lambda_j}{x - x_j} \tag{80}$$

and equivalently for the second barycentric formula.

For the Chebyshev-Lobatto grids, we can derive a closed form representation for the derivative matrix. We already saw the derivative of the Chebyshev polynomials in the previous section, but here can can make use of a nice closed form representation for it, which is more convenient for this derivation:

$$T_k'(x) = kU_{k-1}(x) \tag{81}$$

where U_k are the Chebyshev polynomials of the second kind, which can defined via

$$U_k(\cos(\theta)) = \frac{\sin((k+1)\theta)}{\sin(\theta)} \tag{82}$$

They also from a set of orthogonal polynomials, with respect to the metric $\sqrt{1-x^2}$, i.e.

$$\int_{-1}^{1} dx \sqrt{1-x^2}\; U_n(x) U_m(x) \propto \delta_{nm} \tag{83}$$

To derive now the derivative matrix we can rely on the Chebyshev interpolant written as at the end of the last section:

$$p(x) = \sum_{j=0}^{p} c_j T_j(x), \qquad c_j = \frac{2\gamma_j}{p} \sum_{i=0}^{p} \gamma_i T_i(t_j) f_i \tag{84}$$

from which we have

$$p'(x) = \sum_{j=0}^{p} c_j T_j'(x) = \sum_{j=0}^{p} \left[\frac{2\gamma_j}{p} \sum_{i=0}^{p} \gamma_i T_j(t_i) f_i \right] j U_{j-1}(x) \tag{85}$$

Now we know from the generic discussion that if we evaluate $p'(x)$ on the grid node x_j and we look at the coefficient of the i-th value f_i this gives us the entry D_{ij}. So, rearranging a bit the expression above, we reach the following

$$p'(t_j) = \sum_{i=0}^{p} f_i \left[\sum_{k=0}^{p} \frac{2\gamma_k}{p} k T_k(t_i) U_{k-1}(t_j) \gamma_i \right] \tag{86}$$

To take the sums, one can write all the sine and cosine functions in exponential form. A bit of care is needed for the diagonal terms, and for when either of the i, j indexes is $0, p$. No matter, with patience and the help of Mathematica, one can arrive at the following representation

$$D_{00} = -D_{pp} = \frac{2p^2+1}{6} \tag{87}$$

$$D_{ii} = -\frac{t_i}{2(1-t_i^2)} \tag{88}$$

$$D_{ij} = -\frac{\gamma_i}{\gamma_j} \frac{(-1)^{i+j}}{t_i - t_j} \tag{89}$$

This part has been adapted from the discussion given in [1], with the notable exception that the derivative matrix presented here is the transpose of the derivative matrix of the paper, and notation is slightly different.

8 Non-linear mappings

One of the most useful tools when one wants to perform interpolation out in the real world is the ability of re-mapping the support of the desired variable to a standardized support (which we will assume to be $[-1, 1]$ for definiteness, but it could be different as well). If we are constrained to a linear mapping, this would not be so helpful, beside granting us some code re-usability. Instead, it is the usage of non-linear mapping that really elevates our ability to describe functions for which we know the 'important regions'.

We can produce two, real-world examples where a non-linear mapping is important.

First, the parton distribution functions (which describe the distribution of longitudinal momentum of the constituents of the proton). A general simplified formula could be

$$xf(x) \sim x^{-1/3}(1-x)^5 \tag{90}$$

which is a very simplified form of some of the test Ansätze of [1]. In this context, the domain is $x \in (0, 1]$.

Second, consider the deposited dose by a proton beam in water. A simple parametrization as function of the depth could be

$$\frac{\mathcal{D}(z)}{\mathcal{D}_{\text{ref}}} = \frac{1}{1+\exp\left(\frac{z-R}{\omega}\right)}\left[A + \frac{\exp\left(-\lambda/(R-z+\varepsilon)\right)}{(R-z+\varepsilon)^p}\right] \tag{91}$$

where the domain is $z \in [0, R]$ Where a pseudo-realistic choice for the parameters may be

$$A = \frac{1}{4}, \quad p = \frac{3}{2}, \quad \lambda = \frac{1}{4}, \quad \omega = \frac{1}{2}, \quad R = \frac{45}{2}, \quad \varepsilon = 10^{-4} \tag{92}$$

In Fig.4 they are shown the two examples above, which clearly exhibit the sharp feature that were promised.

In Fig. 3 the same plots, with in red the points corresponding to a 30 nodes for both the Chebyshev-Lobatto grid and an equispaced one.

It is quite clear that with these ranges, equispaced grids cannot realistically capture both the sharp behaviors and the more flat and boring region at the same time. Chebyshev-Lobatto points work better, but again they struggle to correctly follow, in the density of points, the feature of the function.

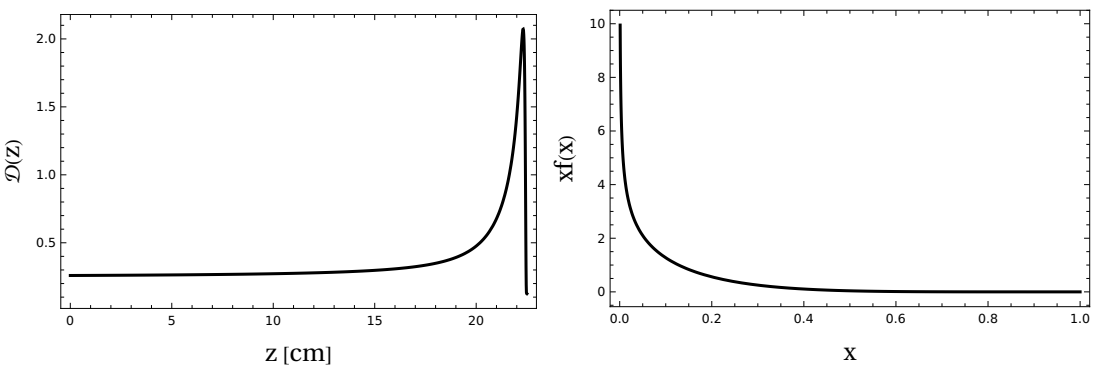

Figure 2: Examples of functions that benefit from non-linear mapping.

8.1 First example

Let us now detail the construction of what is, essentially, the 'industry-standard' mapping that is used for parton distribution functions.

Since the 'sharp feature' in this case is always power-like, and this we can derive from theory, the best mapping is a logarithmic one.

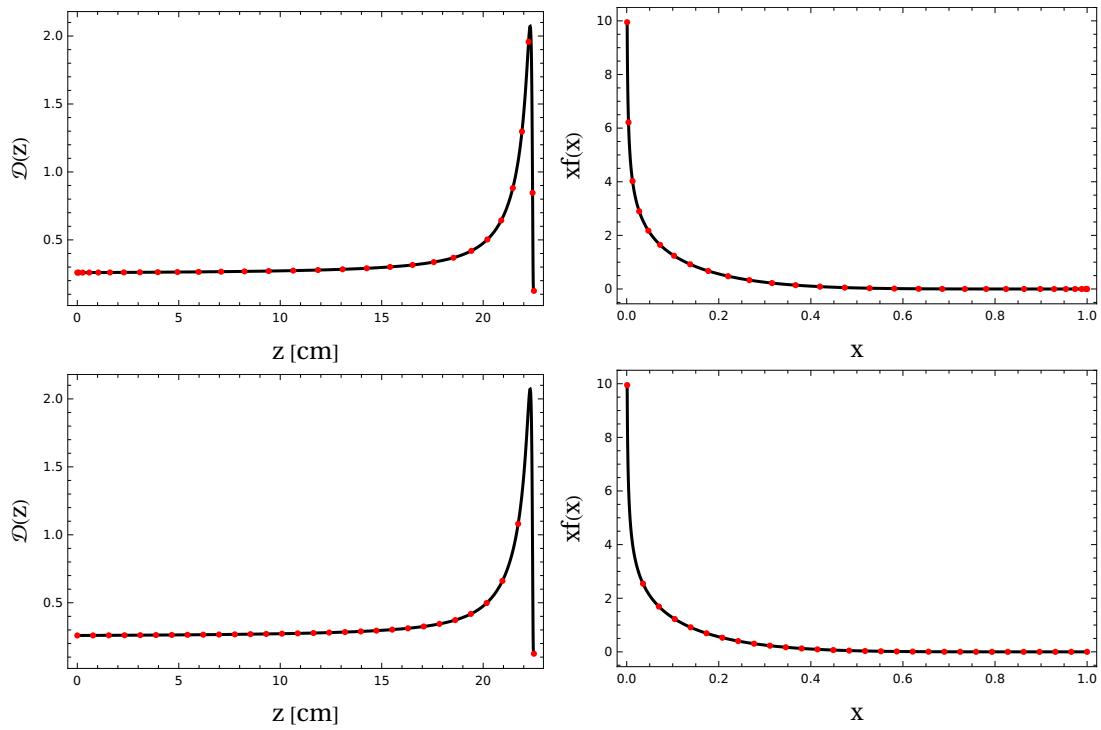

Figure 3: Comparison plot, where in black are the 'exact' functions and in red the points for a grid of 30 nodes. The top panels are with Chebyshev-Lobatto points, the lower ones with equispaced points.

We want to define an intermediate variable u such that

$$u(x) = \log(x) \tag{93}$$

(where log is always the natural log). Since we cannot go to $x = 0$, neither in the original function nor with the variable transformation on top, it is also standard to take the support $x \in [x_0, 1]$, where x_0 is typically small, say 10^{-3} for definiteness (which is what has been used in the plots so far as the lower bound). Now, the support of u is $[\log(x_0), 0]$. Note that, crucially, the function is injective, and that is also monotonically increasing (the first is a requirement, the second just a nice convenient feature). Then, we can introduce the map from u to our normalized space as

$$\tau = 2\frac{u - \log(x_0)}{-\log(x_0)} - 1 \tag{94}$$

One could combine the two, to get a map $x \leftrightarrow t$, but this is not very convenient, since the second linear mapping is actually universal, we just need to know the boundary of u, so, thinking about implementation, we can re-use the linear map for all problems, whereas the original map is problem-dependent.

The two inverse transformations are, trivially

$$u = \frac{\tau + 1}{2}(-\log(x_0)) + \log(x_0) \quad x = \exp(u) \tag{95}$$

Now, we can put a grid in the τ-space: $\{\tau_j\}$ (whether is a Chebyshev-Lobatto grid or something else, doesn't really matter at this stage). This will induce a grid in u-space:

$$u_j = \frac{\tau_j + 1}{2}(-\log(x_0)) + \log(x_0) \tag{96}$$

which in turns induces a grid in x-space, which was our goal

$$x_j = \exp(u_j) \tag{97}$$

8.2 Second example

For the second example, we want more points near the far right end of the support. a reversed log map would work, but for the sake of presenting different maps, let us choose a power-like map, of the form

$$u(z) = z^{\alpha}, \quad z = u^{1/\alpha} \tag{98}$$

Then, the mapping to the normalized space is the same as before, even though now the support of u is $[0, R^{\alpha}]$.

Again the strategy is to put a grid in τ-space, to induce a grid in u-space and, in turn, a grid in z-space.

8.3 Final results

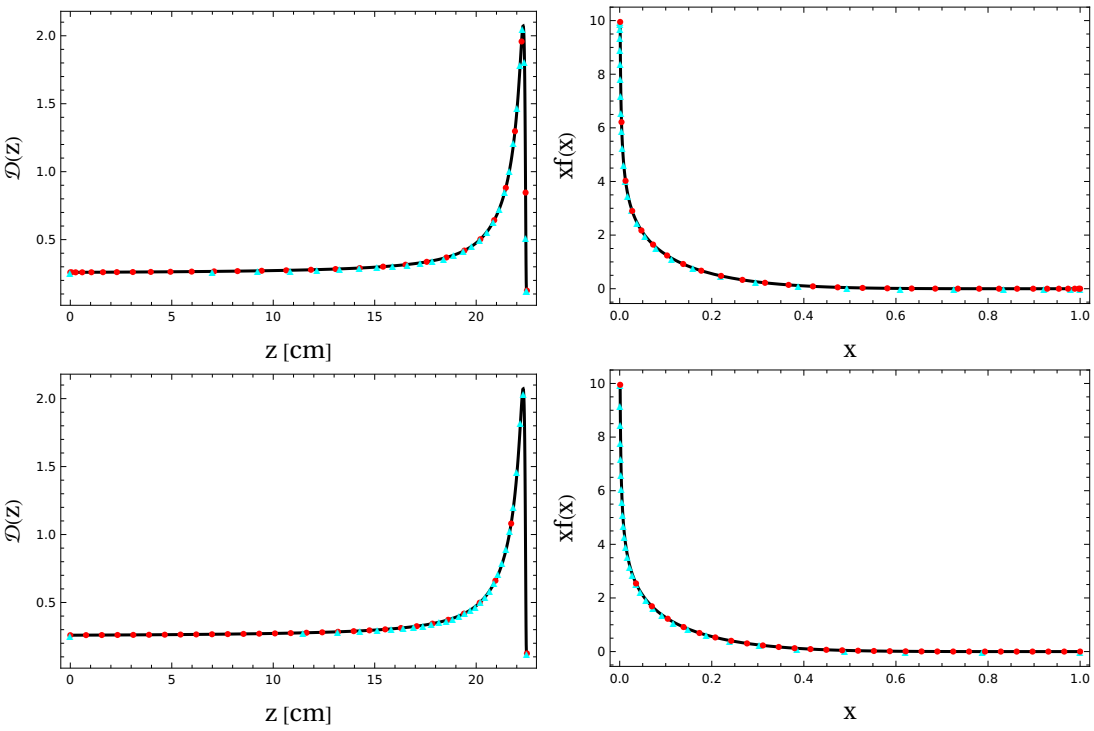

Figure 4: Effects of non-linear mapping. In red circles are the Chebyshev-Lobatto linearly mapped points, whereas in cyan triangles the Chebyshev-Lobatto points with additional non-linear mapping. The bottom panels are the same, but for equispaced grid.

9 Beyond single-grid construction

We saw that, with proper mapping, one can achieve much better densities of the grids where the function has 'important' features. However, we are still limited in what can be achieved in this fashion. One of the most worrying problems is that interpolations with too large polynomial degrees tend to not perform very well (even though some choices are better than others). In practice, constructing polynomial interpolants with degree ≥ 50 is not recommended.

Therefore, we need to find a different approach.

The most versatile alternative, is to construct the global interpolant by a 'collage' of many 'local' ones, under some regularity conditions for the merging points.

We will present to distinct approaches, one that works well for grids like Chebyshev and similar, and the other that relies on the usage of generic grids.

9.1 $\mathcal{C}^0$ tessellation

The simplest, and probably most intuitive, construction is to literally glue together multiple grids, pairwise at the merging points. For instance, say that we want to have a grid that covers $[a, b]$, then we can split the support into $K + 1$ pieces like

$$[a, b] \to [a \equiv x_0, x_1) \cup [x_1, x_2) \cup .. \cup [x_K, b \equiv x_{K+1}] \tag{99}$$

Now, on each interval we can impose a grid (grid_i for the i-th interval), where we assume that the grid_i contains both its extremes points.

To fix the ideas, we can combine in a single map the the non-linear mapping of the previous section and the linear transformation to normalized space. Notice that now each individual grid will have its own linear transformation, since boundaries will be different. We can call the outside variable x, and the inner, standardized variable t. Then, on a single grid, the polynomial interpolant we know we can write as

$$q_i(x) = \sum_{j=0}^{p_i} f(\eta_i(t_j)) b_j(\eta_i^{-1}(x)) \tag{100}$$

where i labels the grid! Since we are requiring that the grids contains both their extreme points, we have a condition on the maps, namely

$$\eta_{i+1}(-1) = \eta_i(1) \tag{101}$$

where we assumed that the normalized space is $[-1, 1]$, in increasing order (this is not the case for the standard Chebyshev-Lobatto grid, for which the points are ordered from $+1$ to -1, but all of what follows applies identically, *mutatis mutandis*)

Now, we can construct the <u>global</u> interpolant as

$$q(x) = \sum_{i=0}^{K} \theta(x \in \text{grid}_i) \sum_{j=0}^{p_i} f(\eta_i(t_j)) b_j(\eta_i^{-1}(x)) \tag{102}$$

where the theta function obviously implements the condition

$$\theta(x \in \text{grid}_i) = \begin{cases} x \in [x_i, x_{i+1}) & \text{if } i \neq K \\ x \in [x_i, x_{i+1}] & \text{if } i = K \end{cases} \tag{103}$$

From now on, to avoid confusion, we will indicate the individual extremes of the intervals via the maps, i.e.

$$\theta(x \in \text{grid}_i) = \begin{cases} x \in [\eta_i(-1), \eta_i(+1)) & \text{if } i \neq K \\ x \in [\eta_i(-1), \eta_i(+1)] & \text{if } i = K \end{cases} \tag{104}$$

Note that we are not requiring that all the grids have the same number of points, in fact we are keeping the freedom of choosing different values of p_i for each i.

Now, Eq. (102) is perfectly fine for interpolation purposes, but often we need to know the individual set of weights associated to each <u>unique</u> grid point. The only common points between adjacent grids are the extreme points, by virtue of Eq. (101). It is therefore instructive to first construct a global grid, that contains only the unique points. Explicitly, we have

$$\left[\bigcup_{i=0}^{K-1} \{\eta_i(t_0), .., \eta_i(t_{p_i-1})\} \right] \cup \{\eta_K(t_0), .., \eta_K(t_{p_K})\} \equiv \{x_0, .., x_{N-1}\} \tag{105}$$

where the total number of unique points N is given by

$$N = 1 + \sum_{i=0}^{K} p_i = -K + \sum_{i=0}^{K} N_i \tag{106}$$

where $N_i = p_i + 1$ is the number of points in the i-th grid, and the last expression makes it clear that we are removing the exactly K common points.

Therefore, our vector of unique function values can be computed on the points x_i. So, we aim to write

$$q(x) = \sum_{l=0}^{N-1} f(x_l) w_l(x) \tag{107}$$

With a little bit of simple algebra, we reach the following representation for the <u>global</u> weights w_l:

$$w_l(x) = \theta(x \in \text{grid}_{i(l)}) b_{j(l)} \left(\eta_{i(l)}^{-1}(x)\right) + \delta_{i(l)\neq 0}\delta_{j(l)=0}\theta(x \in \text{grid}_{i(l)-1}) b_{p_{i(l)-1}-1} \left(\eta_{i(l)-1}^{-1}(x)\right) \tag{108}$$

Horribly verbose... The meaning is although quite clear: for any given index l we know how to compute the underlying indexes $i(l), j(l)$, where the first labels the grid and the second the point on the grid, because we constructed the index l starting from i, j in the first place. Then, we have the first contribution, that is just the natural term in the interpolation formula. If however we are looking at the first point of a subgrid ($\delta_{j(l)=0}$), but avoiding the case of the very first subgrid ($\delta_{i(l)\neq 0}$) which has no subgrid on its left, then we need to include the interpolating polynomial that corresponds to it, since we are storing intervals that are closed on the left and open on the right (if we were to do the opposite, we would need to check if we are sitting at the right-most point of a subgrid).

This final representation also makes it evident that, by construction, the global interpolating function is continuous at the boundary of subgrids. It cannot, however, guarantee any differentiability requirement, because the interpolant from the left and the right of a grid-joining point can reach the point with different slope. This might seem bad, but it is actually a very desirable feature: it means that with this construction we can interpolate functions with discontinuous first derivative at few (known) points. If the function has a discontinuity in the first derivative, and we would try to interpolate it with a smooth polynomials, we would loose a lot of precision.

Conceptually, the construction above does what we want. However, thinking about implementation, this is not ideal. From a code point of view, when we construct and store our joint grid, we can be a little bit smarter.

The global weights are clearly composed of two separate contributions. Therefore, we can define the weights as the individual b_j for each grid in a flatten vector:

$$\tilde{w}_n(x) = \theta(x \in \text{grid}_{i(n)}) b_{j(n)} \left(\eta_{i(n)}^{-1}(x)\right) \tag{109}$$

where the theta function does not need to be included, but can be stored separately since it statically encode the support of the given weight. Now the index n runs over all the points of the grid, including duplicates! We have, however, a non-trivial mapping between the index n and the index l, and this mapping is required to know how to associate each value $f(x_l)$ to its set of weights, and viceversa.

By construction, it should be clear that the map $n \to l$ is 1-to-1, i.e. for a given n, there is only one associated l. The contrary is not true, i.e. $l \to n$ is, in general, 1-to-many. In practice, $l \to n$ is almost always 1-to-1, with the exceptions of the indexes l that satisfy the deltas $\delta_{i(l)\neq 0}\delta_{j(l)=0}$, in this case the map is 1-to-2, i.e. the given l has two associated n values.

It is actually much harder to describe the algorithm in words than to give it to you explicitly, therefore here it is:

```
c_size ← N     compute the total number of unique points
n ← 0
l ← 0
for a ← 0 to K do
    Save current index as the delimiter of two subgrid
    _delim_indexes[a] ← n
    Loop over all points of the a-th grid
    for j ← 0 to grid_deg[a] do
        if j ≠ grid_deg[a] or a = K then
            Compute coordinate in intermediate u space by linear map
            _coord_inter[l] ← lin_map(tj, lims[a].first, lims[a].second)
```

```
            Compute the coordinate in physical x space
            _coord[l] ← to_phys_space(_coord_inter[l])
        end if
        Fill map for n → l
        _from_iw_to_ic[n] ← l
        Fill map for n → i(n), i.e. from index to interval index
        _from_idx_to_inter[n] ← a
        If we are not on the right-most point of an interval, advance l index
        if j ≠ grid_deg[a] then
            l ← l + 1
        end if

        Construct the weights at index n...

        Advance index n
        n ← n + 1
    end for
end for
Append last point to delimiter vector
_delim_indexes[K + 1] ← n
Resize _from_ic_to_iw to size c_size
Loop over all entries in the map n → l
for n ← 0 to size(_from_iw_to_ic) − 1 do
    get l that corresponds to specific n
    l ← _from_iw_to_ic[n]
    Append n to _from_ic_to_iw[l]
end for
```

In this algorithm, the map from n to l `_from_iw_to_ic` is constructed as a plain list of values, one for each n.

The inverse map `_from_ic_to_iw`, from l to n, is constructed as a list of lists, one list for each l that contains all the corresponding n values.

The weights themselves are a list (of functions) indexed by n, whereas the coordinates in u and x space (`_coord_inter` and `_coord` respectively) are a list (of values) indexed by l.

In the pseudo-code above, I am omitting some logic to get the correct values t_j for the a-th sub-grid that is being considered. This is a detail (albeit a practically important one), that it will be clarified in the coding sessions.

We can also convince ourselves quite easily that this whole construction is indeed C^0 at the merging points. Let us assume that we have only two sub intervals, and that on each we have just two points, therefore the full grid looks like $[x_0, x_1, x_2]$, with x_1 being the merging point. By construction, on each of the two sub-intervals, we have a linear interpolation, since only two points are available.

Therefore, we can quite easily write the full interpolant as

$$q(x) = \theta(x \in [x_0, x_1)) \left[\frac{x - x_0}{x_1 - x_0}(f_1 - f_0) + f_0 \right] + \theta(x \in [x_1, x_2]) \left[\frac{x - x_1}{x_2 - x_1}(f_2 - f_1) + f_1 \right] \tag{110}$$

Now, we immediately have that

$$q'(x) = \theta(x \in [x_0, x_1)) \frac{f_1 - f_0}{x_1 - x_0} + \theta(x \in [x_1, x_2]) \frac{f_2 - f_1}{x_2 - x_1} \tag{111}$$

and, unless we are interpolating exactly a linear function, we can never find continuity of the first derivative at the boundary, since in general

$$\frac{f_1 - f_0}{x_1 - x_0} \neq \frac{f_2 - f_1}{x_2 - x_1} \tag{112}$$

Notice that we never considered the derivatives of the theta function. In general, if you are willing to interpret the interpolant as a generalized function (aka distribution) you may try to derive the theta function, obtaining a Dirac delta function, but then one looses completely the ability of representing such an object on a computer, unless one is solely interested in considering integral convolutions.

Therefore, in this case, the theta function is just a convenient short-hand way to represent the fact that we use either term of the sum depending on which sub-interval we happen to fall into.

9.2 Sliding window

The tessellation just presented is not the only way. We could consider pushing the concept to the extreme and use as many intervals as we have points (minus one, of course). However, this would impose a piece-wise linear interpolation, which is not ideal for precision or speed.

But what if we can still have one unique interpolant for each range $x \in [x_\alpha, x_{\alpha+1}]$, but without having to rely on linear interpolation? The construction showed in the following is based upon the following references [2, 3].

The idea is to have a grid (that in realistic settings cannot really be a Chebyshev-Lobatto grid or similar) in the variable x, which we will label with greek indexes, that spans the whole support that we are interested in from the beginning. Of course such a grid can be induced by a grid in another space, say u space, via some non-linear mapping, but this is just a detail, which is irrelevant for the construction itself.

Once we have the grid, we can construct the local modes of interpolation based on the relative location of x on the grid. Specifically, assuming that $x \in (x_\alpha, x_{\alpha+1}]$, we have that

$$q_\alpha(x) = \sum_{i=0}^{k} l_i(x; \{x_\alpha, .., x_{\alpha+k}\}) f(x_{\alpha+i}) \tag{113}$$

where we went back to show the explicit set of nodes that are used in the construction of the Lagrange interpolants. To be thorough, we can write the inteprolant explicitly in the naïve representation as

$$l_i(x; \{x_\alpha, .., x_{\alpha+k}\}) = \prod_{\substack{j=0 \\ j \neq i}}^{k} \frac{x - x_{\alpha+j}}{x_{\alpha+i} - x_{\alpha+j}} \tag{114}$$

Now we need to construct the global interpolation modes, which are just the sum of the local ones, with appropriate theta functions:

$$q(x) = \sum_{\alpha=0}^{p} \theta(x \in (x_\alpha, x_{\alpha+1}]) q_\alpha(x) \tag{115}$$

Notice however that the last few elements of this sum present a problem, specifically we would need to consider points up to subscripts x_{p+k}, which seems to be undefined. Indeed this is the major downside of this approach, that is one needs to extend the grid beyond the original range, by however many points are needed to never access an index that is out-of-bound. In practice, interpolation degree for this approach is never large, typically $k = 1 \sim 5$ in most application works well. Therefore, one needs only a handful of additional points. The interpolation support remains $[x_0, x_p]$, otherwise we would need to construct grids that have infinitely many points. The additional points pose the problem of what to do with the function evaluation on those points. Sometimes it makes perfect sense to evaluate the function outside the chooses domain, other times it does not. When it does not, one typically extends the function to have the same value on the extra nodes as the value that it took on the very last node, but whether or not this is a good approximation really depends on the problem under consideration.

Leaving aside these considerations for a moment, we have not yet fully derived the global modes, which are defined by being the coefficients of each unique f_α evaluations.

To reach the global mode representation, we can re-arrange the indexes, by introducing $\beta = \alpha + i$. This transforms the two independent sums into dependent ones (which seems the opposite of what one wants

to do), but it does the trick:

$$q(x) = \sum_{\beta=0}^{p+k} f(x_\beta) \underbrace{\sum_{i=0}^{\min(\beta,k)} \theta(x \in (x_{\beta-i}, x_{\beta-i+1}])l_i(x; \{x_{\beta-i}, .., x_{\beta-i+k}\})}_{w_\beta(x)} \tag{116}$$

where $w_\beta(x)$ are the sought after global weights. It is also simple to derive their support from the theta functions, namely

$$w_\beta(x) \neq 0 \quad \text{if } x \in (x_{\beta-\min(\beta,k)}, x_{\beta+1}] \tag{117}$$

We can also deduce from this that only a limited number of weights will in practice participate in the sum of the interpolant. The global modes are nothing more than a combination of local modes. There will be only one local mode contributing for any given value of x, by the very definition of the construction. Such local mode will have inside all the evaluations of the function at $x_\alpha, .., x_{\alpha+k}$. Since these evaluation points exactly identifies one global mode, we immediately have that the global mode representation of the interpolant can be condensed to

$$q(x) = \sum_{\beta=\alpha(x)}^{\alpha(x)+k} f(x_\beta) w_\beta(x) \tag{118}$$

where $\alpha(x)$ is defined as the (unique) index for which $x \in (x_\alpha, x_{\alpha+1}]$.

9.2.1 Further generalization

We can now push this approach even further, by generalizing the construction to interpolate not when $x \in (x_\alpha, x_{\alpha+1}]$, but rather when $x \in (x_{\alpha+t}, x_{\alpha+t+1}]$.

At this point it should be clear that the construction in local and global modes are just two different ways to present the same exact interpolant. We can formalize this by introducing a matrix representation for the construction. Specifically, let us construct

$$\mathbb{L} = \begin{pmatrix} l_0(x;(0,k)) & l_1(x;(0,k)) & .. & l_k(x;(0,k)) & 0 & .. & 0 \\ 0 & l_0(x;(1,k+1)) & .. & .. & l_k(x;(1,k+1)) & 0 & .. \\ .. & .. & .. & .. & .. & .. & .. \\ 0 & .. & .. & .. & .. & 0 & l_k(x;(p,k+p)) \end{pmatrix} \tag{119}$$

where I used

$$l_i(x;(\alpha,\beta)) = l_i(x; \{x_\alpha, .., x_\beta\}) \tag{120}$$

to shorten the notation. The matrix $\mathbb{L}$ is clearly a $(p+1) \times (p+k+1)$ matrix. Now, introduce the vector $\boldsymbol{f}$ of dimension $p+k+1$ which is defined by

$$\boldsymbol{f} = (f(x_0), .., f(x_{p+k}))^T \tag{121}$$

and the vector $\boldsymbol{\theta}$ of dimension $p+1$

$$\boldsymbol{\theta} = (\theta(x \in [x_0, x_1]), \theta(x \in (x_1, x_2]), .., \theta(x \in (x_{p-1}, x_p]))^T \tag{122}$$

Now, we can represent the interpolant in the following matrix representation

$$q(x) = \boldsymbol{\theta}^T \mathbb{L} \boldsymbol{f} \tag{123}$$

Now the local mode representation is just obtained by explicitly first taking the $\mathbb{L}\boldsymbol{f}$ product, and then the product with $\boldsymbol{\theta}^T$, whereas the global mode representation is obtained by first taking $\boldsymbol{\theta}^T\mathbb{L}$ and then the product with $\boldsymbol{f}$.

It should be clear by the representation that we can change each individual row of the matrix as we see fit, with the caveat that we need to maintain a non-block-structure for the matrix, i.e. we cannot identify

two sets of rows that have no overlapping non-zero entries, otherwise we would not guarantee that the construction is at least C^0.

Note that a similar matrix-like construction can be applied to the tessellation construction, where now we have way fewer rows in the matrix. Specifically, if we have $K+1$ sub-intervals, the matrix will have $K+1$ rows, and N (= total number of unique points) columns. For the example of two sub-intervals, with two points each, the explicit construction for the matrix would be

$$\begin{pmatrix} \frac{x_1-x}{x_1-x_0} & \frac{x-x_0}{x_1-x_0} & 0 \\ 0 & \frac{x_2-x}{x_2-x_1} & \frac{x-x_1}{x_2-x_1} \end{pmatrix} \tag{124}$$

More in general, the matrix for the tessellation construction will look like

$$\begin{pmatrix} \bullet & .. & \bullet & 0 & .. & 0 & .. & .. \\ 0 & .. & \bullet & \bullet & .. & \bullet & 0 & .. \\ 0 & .. & 0 & 0 & .. & \bullet & \bullet & .. \end{pmatrix} \tag{125}$$

where between each row there is exactly one column that is shared.

For the general case of the 'sliding window' approach, we also note that for the given construction we will never have differentiability at any of the grid points! This can be understood by going back to the original intuition and motivation of having this approach as the limit of having as many sub-intervals as we have grid points. In between the points the functions are of course differentiable. And we also know how to construct the derivative, since we can always go back to the local mode representation, which is nothing more that a standard combination of Lagrange interpolants, and construct the derivative matrix for the subset of points. Then, since we have the representation of the global modes in terms of the local ones, we can always construct the derivative of the global modes.

10 Integration on the grid

Since we talked about interpolating a function, and we illustrated a way to compute the derivative of a function from the polynomial interpolant, it is only natural to ask if the same ideas could be apply to compute the integral of a function via the polynomial interpolant. The answer is of course yes, and what we will discuss in this section forms the basis of how one can take a large class of integrals numerically.

For the moment our focus is on the direction of thought: assign f on some domain, compute the polynomial interpolant, compute the integral of the interpolant, obtain an estimate of the integral of f on the support.

We start, as always, from the Chebyshev polynomials.

We recall that we can derive the interpolant for the Chebyshev-Lobatto grids in terms of the Chebyshev polynomials as

$$p(t) = \sum_{j=0}^{p} c_j T_j(t), \qquad c_j = \frac{2\gamma_j}{p} \sum_{i=0}^{p} \gamma_i T_i(t_j) f_i \tag{126}$$

Let us forget about additional mappings to begin with. The formula above can be re-arranged to be

$$p(t) = \sum_{i=0}^{p} f_i b_i(t), \qquad b_i(t) = \sum_{j=0}^{p} \frac{2\gamma_j\gamma_i}{p} T_i(t_j) T_j(t) \tag{127}$$

Now, we want to study

$$\int_{-1}^{1} dt\ p(t) = \int_{-1}^{1} dt\ \sum_{i=0}^{p} f_i b_i(t) \tag{128}$$

Since the sum is a finite one, and not a series, we can simply exchange it with the integration, to reach

$$\sum_{i=0}^{p} f_i \int_{-1}^{1} dt\ b_i(t) = \sum_{i=0}^{p} f_i w_i \sim \int_{-1}^{1} dt\ f(t) \tag{129}$$

The integration weights are simple to compute from the fundamental integral

$$\int_{-1}^{1} dt\ T_j(t) = \int_0^{\pi} d\theta\ \sin(\theta)\cos(j\theta) = \begin{cases} \frac{2}{1-j^2} & \text{if } j \text{ is even} \\ 0 & \text{otherwise} \end{cases} \tag{130}$$

from which

$$w_i = \int_{-1}^{1} dt\ b_i(t) = \sum_{k=0}^{[\![p/2]\!]} \frac{4\gamma_{2k}\gamma_i}{p} T_i(t_{2k}) \frac{1}{1-4k^2} \tag{131}$$

where $[\![p/2]\!]$ is the integer division of p by 2 (i.e. $p/2$ if p is even $(p-1)/2$ if p is odd).

What about the case where we want to include a non-linear mapping? nothing really changes, that is we have

$$p(x) = \sum_{i=0}^{p} f(\eta(t_i)) b_i(\eta^{-1}(x)) \tag{132}$$

and now we must compute

$$\int_a^b dx\ p(x) = \int_{-1}^{1} dt\ \eta'(t)\ p(\eta(t)) = \sum_{i=0}^{p} f(\eta(t_i)) \int_{-1}^{1} dt\ \eta'(t) b_i(t) \tag{133}$$

which is now obviously subordinated to the ability of taking the right most integral analytically, or to the ability of taking it numerically.

For the moment, we state that a good-enough approximation for the computation (which should be used only after scrutiny of stability) is to approximate

$$\sum_{i=0}^{p} f(\eta(t_i)) \int_{-1}^{1} dt\ \eta'(t) b_i(t) \simeq \sum_{i=0}^{p} f(\eta(t_i))\eta'(t_i) \int_{-1}^{1} dt\ b_i(t) = \sum_{i=0}^{p} f(\eta(t_i))\eta'(t_i) w_i \tag{134}$$

What about the general case? In this context we will not have a clean single-expression formula, but we can still work it out. It should be evident that we can rewrite the Lagrange interpolants on the basis of monomials as

$$l_j(x) = \lambda_j \prod_{\substack{i=0 \\ i\neq j}}^{p} (x - x_i) = \lambda_j \sum_{k=0}^{p} x^k c_k^{(j)} \tag{135}$$

from which

$$\int_a^b dx p(x) = \sum_{j=0}^{p} f_j \sum_{k=0}^{p} c_k^{(j)}\ \frac{b^{k+1} - a^{k+1}}{k+1} \tag{136}$$

where we used the trivial relation

$$\int_a^b dx x^k = \frac{b^{k+1} - a^{k+1}}{k+1} \tag{137}$$

Part II

Numerical Quadrature

With most of the basics about polynomial interpolations done, we have the stage prepared to study the problem of integrating a function of an interval. This problem is typically referred to as quadrature.

In general, we want a way to compute the integral

$$\int_a^b dx\ f(x) \equiv I \tag{138}$$

to within a certain absolute and relative accuracy. Any numerical routine that we will construct to solve the quadrature problem will give us a pair of numbers of the form $\{R, E\}$, where R is our estimation of the result, and E the estimation of the error. We want to achieve the converge condition

$$|R - I| \leq \max(\varepsilon_a, \varepsilon_r |I|) \equiv \varepsilon \tag{139}$$

where ε_a is the absolute accuracy and ε_r is the relative one. Note that in the definition above we state that the result is satisfactory if we can reach either the absolute or the relative accuracy requirement. Stronger statement would be to require that we reach both, for which the max would get replaced by a min. We also want that

$$|R - I| \leq E \leq \varepsilon \tag{140}$$

i.e. that our estimation of the error is always conservative (i.e. lower than the requested accuracy) and always accurate (i.e. we never declare an error that is smaller than the actual error that we have).

This is typically achieved by a sequence of evaluations of pairs $\{R_i, E_i\}$, which are increasingly more accurate. Any algorithm will only generate so many pairs before either reaching convergence Eq.(139) or aborting if, after some max number of pair generation, the required accuracy cannot be reached.

There should be some intuition on the fact that, to generate multiple pairs of results and error estimate, any algorithm will need to sample a certain number of points. We can divide any, non-statistical, algorithm into two main classes: adaptive and non-adaptive. The former typically refers to a class of algorithm that will keep bisecting the original intervals, generating the estimation of the result and error as the sum of the results and errors for each sub-interval. The latter refers to an algorithm that will never bisect the interval, but rather increase the number of points used in the evaluation.

In both scenarios, the algorithm has a rule for calculating an integral approximation together with a pertinent error estimate. These are dubbed local integral approximation and local error estimate.

The adaptive version has components that will decide how and when to continue integration by bisection. In the adaptive version is also possible to include components for detecting and handling (integrable) singularities. The non-adaptive version will instead stop either if it reaches a maximum number of pair generation, or it reaches convergence.

For the adaptive algorithm, one further distinguish two sub-classes: locally-adaptive and globally-adaptive. In a locally-adaptive algorithm, the convergence criterion is imposed on each bisected interval, with no guarantees on the global accuracy of the result. In a globally-adaptive algorithm, the viceversa is true, i.e. the convergence criterion is applied on the global error estimate, but not locally on each bisected interval.

We will also distinguish general-purpose integration routines, which as the name implies can be applied to any generic problem and specific-purpose integration routines. The latter are specialized for a particular quadrature problem, and typically out-perform general-purpose one on the specific problem. They cannot however be applied for a generic problem. For special-purpose routines, we will only look at one, that is of capital importance in physics, which is how to take the integral

$$\int_0^\infty dx\ J_\nu(x) f(x) \tag{141}$$

where J_0 is the Bessel function of the first kind. The algorithm itself is called Ogata-quadrature, which is named after its inventor.

11 Gauss formulas

Now, you should have built some intuition from the last section of the polynomial interpolation to be able to guess that the underlying tools to compute integrals by quadrature are polynomial interpolation formulas.

Let us take the problem of computing

$$I_w^{[a,b]}[f] = \int_a^b dx\ \omega(x) f(x) \tag{142}$$

where for the moment the interval is finite (i.e. $-\infty < a < b < \infty$), and $\omega(x)$ is some known weight function. Now, let us introduce <u>an</u> interpolant for f on a grid between a and b:

$$f(x) \sim p(x) = \sum_{i=0}^{p} f_i b_i(x) \tag{143}$$

Then, we can obtain a quadrature estimate for the integral from

$$Q_{w,p}^{[a,b]}[f] = \sum_{i=0}^{p} w_i f_i \tag{144}$$

where the integration weights are defined via

$$\int_a^b dx\ \omega(x) b_i(x) = w_i(a,b) \equiv w_i \tag{145}$$

and in general depends on the given interval under consideration.

A useful restriction on the weight function $\omega(x)$ is that it is a real function (for complex integrals, different techniques ought to be used) and that has well defined all the integer integral moments

$$\mu_n = \int_a^b dx\ \omega(x) x^n \tag{146}$$

for any $n \geq 0$.

An important property of any quadrature formula is its polynomial degree, which is not quite the degree of the interpolating polynomial, but rather the maximal polynomial degree for which the quadrature formula is exact, meaning that the difference between the quadrature result and the exact result vanishes.

Now, in the presentation above I cheated a little bit, in the sense that one could also consider to put grid nodes <u>outside</u> the interval $[a, b]$, and one may not necessarily think of the weights w_i as the integrals of the interpolating polynomial. However, in practice, one almost always wants integration nodes that lie in the original interval (the integrand could be very easily meaningless outside the interval) and, a general desirable property for the integration weights is that they are all positive. This is because having positive and negative weights could lead to large numerical cancellations, which would render any algorithm potentially unstable.

If one constructs the integration weights from the polynomial interpolant, one immediately realizes that the given quadrature formula (i.e. the joint collection of nodes and weights) is of polynomial degree p. This is by construction, because we know that the interpolant is the unique polynomial of the given degree that satisfies the interpolation constraint, and therefore it is just a different representation for the function if the function itself is a polynomial of appropriate order. Such rules are known as interpolatory rules.

We can also push the construction further, by choosing the nodes of integration is a very specific way, dictated by the weight function $\omega(x)$, to reach the so-called Gauss rules. Gauss rules are still interpolatory rules, but with the added benefit that they can integrate exactly polynomials up to polynomials of degree $2p+1$.

Because we assumed that the weight function has all integer integral moments, and that the weight function is real, if we add the constraint that $\omega(x) \geq 0$, effectively the weight function becomes an integral measure, and we know that once we assign an integral measure, we can always find a family of polynomials that is orthogonal to that measure and that form a basis for the polynomial space. For the sake of giving them a name, let us call the polynomial of the basis $\mathfrak{p}_n$

Therefore, we choose the integration nodes to be the zeros of the $\mathfrak{p}_{p+1}$ polynomial. From classical polynomial theory, we know that any polynomial of degree $2p+1$ can be decomposed as $\pi(x) = \mathfrak{p}_{p+1}(x)q_p(x) + r_p(x)$, where q_p, r_p are two polynomials of degree $\leq q$. Now, the integral of $\pi(x)$ can be written as

$$\int_a^b dx\ \omega(x)\pi(x) = \int_a^b dx\ \omega(x)\mathfrak{p}_{p+1}(x)q_p(x) + \int_a^b dx\ \omega(x)r_p(x) \tag{147}$$

Since our polynomials $\mathfrak{p}$ are an orthogonal basis, the first integral vanishes, because can always be casted into a sum of terms of the form $\langle \mathfrak{p}_{p+1} | \mathfrak{p}_n \rangle_\omega$ for $n \leq p$. Therefore we have that

$$\int_a^b dx\ \omega(x)\pi(x) = \int_a^b dx\ \omega(x)r_p(x) \tag{148}$$

If we now choose an interpolatory formula with $p+1$ points, the right hand side is integrated exactly (by the very definition of interpolatory). If we also add the requirements that our $p+1$ points are exactly the zeros of $\mathfrak{p}_{p+1}$, the quadrature rule becomes

$$\sum_{i=0}^{p} w_i\pi(x_i) = \sum_{i=0}^{p} w_i\mathfrak{p}_{p+1}(x_i)q_p(x_i) + \sum_{i=0}^{p} w_i r_p(x_i) = \sum_{i=0}^{p} w_i r_p(x_i) = \int_a^b dx\ \omega(x)r_p(x) \tag{149}$$

We just proved that, under a few special requirements, we can always integrate exactly any polynomial up to degree $2p+1$ using $p+1$ integration nodes.

Probably the most famous case is the Gauss-Legendre quadrature. In this case, the interval is fixed by $a=-1$, $b=+1$, and we recall that the Legendre polynomials are orthogonal with respect to the unit measure, i.e. $\omega(x)=1$. We can give few more examples, which you can also find in [4].

a	b	ω	Polynomial basis
-1	$+1$	1	Legendre
-1	$+1$	$1/\sqrt{1-x^2}$	Chebyshev T
-1	$+1$	$\sqrt{1-x^2}$	Chebyshev U
-1	$+1$	$(1-x)^\alpha(1+x)^\beta$	Jacobi P, for $\alpha, \beta > -1$
0	∞	e^{-x}	Laguerre
0	∞	$x^\alpha e^{-x}$	Generalized Laguerre
$-\infty$	∞	e^{-x^2}	Hermite

Let us now focus on how to explicitly construct the integration weights, once we identified the proper interval and polynomial basis. We will use Legendre, simply for convenience, but all similar considerations apply for all cases. We have already deduced that for any interpolatory formula the integration weights are nothing but the integrals of the basis functions.

We also recall that the basis functions can be written in the first barycentric formula as

$$b_i(x) = l(x)\frac{\lambda_i}{x-x_i} = \frac{l(x)}{l'(x_i)}\frac{1}{x-x_i} \tag{150}$$

where we used the definition of the interpolating weights as derivatives of the monic polynomial $l(x)$. Now, the monic polynomial was defined as the polynomial with exactly $p+1$ zeros in correspondence with the grid nodes. Therefore, by construction, this is proportional to $L_{p+1}(x)$ (or the $p+1$ polynomial

in the appropriate basis for other cases). The proportionality constant is just one over the coefficient of x^{p+1} of $L_{p+1}(x)$. The constant is irrelevant, since it cancels in the ratio $l(x)/l'(x_i)$. Therefore, we have that

$$\frac{1}{L'_{p+1}(x_i)} \int_{-1}^{1} dx \frac{L_{p+1}(x)}{x - x_i} = w_i \tag{151}$$

We will not discuss techniques to derive the explicit result of this type of integrals. You can typically find them tabulated. Also, we completely glossed over on how to determine the zeros of the given set of polynomials that we are interested in. This is not always an easy task, especially if you want them for any arbitrary degree. Again, in most cases you can find them tabulated.

12 Rule extensions and error estimate

Let us note that we obtain a very powerful tool to compute an estimation of the result R, but we have never discussed how to obtain an estimation for the error E! The first solution that one may think of trying is to just compute a result estimation with p and $p+1$ points, and use their difference as an estimation for the error. This works, but has the major disadvantage that it requires $2p+1$ function evaluations, since in general a rule with p and $p+1$ points have no points in common. Moreover, a refinement of just one point may not be enough to obtain a reliable estimation for the error, and typically one wants to be more conservative. The idea would be to double the number of points, but this is even worse, since it will require $3p$ evaluations of the functions. Our goal now is to find a way to extend any given Gauss formula (and we will do it for the Gauss-Legendre formula only) so that we can achieve a result with a high degree of precision, together with a reliable error estimation.

In the context of Gauss-Legendre quadrature, such formula does exist, and is typically known as the Gauss-Kronrod formula. In reality they are a family of formulas, one for each underlying Gauss formula.

The idea is to start with a Gauss-Legendre formula with m integration nodes (which should integrate exactly polynomials up to degree $2m-1$), and add to it an extra $m+1$ Kronrod nodes, which represent the optimal extension to reach exact integration of polynomials up to degree $3m+1$. Note that we cannot reach polynomial degree $4m+1$ with this $2m+1$ rule, otherwise it would not be an extension of the Gauss rule, but rather a Gauss rule itself with $2m+1$ nodes, which we know shares no points with the lower m node formula.

We will not present the derivation, which is long (albeit instructive), and can be found in [4].

We will just comment on the final general structure. First, the integration nodes will be interleaved between pure Gauss nodes, and the nodes for the optimal extension. Second, because of the properties of Legendre polynomials, we will always have that the nodes (and associated weights) come in pairs $\pm x_i$, and the origin is present for the Gauss rule only if it has an odd number of integration nodes, whereas it is always present in the combined set of nodes, since the combined rule always as an od number of nodes.

Then, especially for the Gauss-Legendre rules, it is trivial to change the integration interval from being just $[-1,1]$ to the generic $[a,b]$ via the linear transformation

$$\int_a^b dx\ f(x) = \frac{2}{b-a} \int_{-1}^{1} dt\ f\left(\frac{b-a}{2}t + \frac{b+a}{2}\right) \tag{152}$$

Therefore, we can always store and compute the integrals as if the interval is $[-1,1]$.

The following is a stripped down version of a pseudo code for the computation of an integral and corresponding error estimate using Gauss-Kronrod:

Require: Interval endpoints a, b, integrand f, Gauss weights $w^{(G)}$, Kronrod weights $w^{(K)}$, Gauss-Kronrod nodes $x^{(K)}$, rule order n

1: Computes Gauss estimate I_G and Kronrod estimate I_K
2: $c \leftarrow \dfrac{a+b}{2}$, $h \leftarrow \dfrac{b-a}{2}$, $f_c \leftarrow f(c)$
3: $I_G \leftarrow 0$
4: $I_K \leftarrow w^{(K)}_{\text{center}} f_c$

if n is even **then**
    Add the center contribution to the Gauss estimate:
    $I_G \leftarrow I_G + w^{(G)}_{\text{center}} f_c$
end if
for all Kronrod nodes that also belong to the Gauss subset, excluding the center **do**
    $x \leftarrow h \cdot x_i$
    Evaluate for both positive and negative nodes:
    $s \leftarrow f(c-x) + f(c+x)$
    Accumulate the contribution into both rules:
    $I_G \leftarrow I_G + w_i^{(G)}\, s$, $I_K \leftarrow I_K + w_i^{(K)}\, s$
end for
for all Kronrod nodes that are not Gauss nodes **do**
    $x \leftarrow h \cdot x_i$
    Evaluate symmetrically:
    $s \leftarrow f(c-x) + f(c+x)$
    Accumulate only in the Kronrod estimate:
    $I_K \leftarrow I_K + w_i^{(K)}\, s$
end for
Rescale both sums from the reference interval to $[a, b]$:
$I_G \leftarrow h\, I_G, \qquad I_K \leftarrow h\, I_K$
return Result estimate I_K, Absolute error estimate $|I_K - I_G|$, Relative error estimate $|1 - I_G/I_K|$

Note that the error estimate in this case is typically very conservative, i.e. in most if not all practical cases both the absolute and relative error estimates are much larger than the actual error.

Let us now take a little detour to study an ad-hoc quadrature formula, before returning to Gauss-like rules using adaptive integrations.

13 Clenshaw-Curtis rules

A special case of a non-Gauss rule that still performs incredibly well is the integration performed using as nodes the Chebyshev-Lobatto points. We already saw the derivation of the integration weights at the end of the last part, and here they are again

$$w_i = \sum_{k=0}^{[\![p/2]\!]} \frac{4\gamma_{2k}\gamma_i}{p} \cos\left(\frac{2ki\pi}{p}\right) \frac{1}{1-4k^2} \tag{153}$$

and the integral is estimated by

$$\int_{-1}^{1} dt f(t) \sim \sum_{j=0}^{p} f(t_j) w_j \tag{154}$$

There is no direct way to extract an error estimate from this formula. However, one can take out a page from the Gauss-Kronrod book to try to come up with another rule, that re-use some of the points, to construct the error estimate as the difference between the Clenshaw-Curtis rule and the other rule. In this context there is a quite natural second rule that one can use, that is derived from the Chebyshev polynomials of the second kind.

We already encountered these polynomials when we discussed the derivatives of the Chebyshev interpolant.

Through a similar procedure to what we used for the Chebyshev interpolant, one can derive the following representation for the interpolation of a function on the basis of Chebyshev polynomials of the second kind:

$$q(x) = \sum_{k=1}^{p-1} d_k U_{k-1}(x) \tag{155}$$

where

$$d_k = \frac{2}{p}\sum_{j=1}^{p-1} f(t_j)(1-t_j^2)U_{k-1}(t_j) \tag{156}$$

One can verify that $q(t_j) = f(t_j)$ for $j = 1, .., p-1$ using Eq.(82).

Following identical steps to before, one obtains that

$$\int_{-1}^{1} dt U_{j-1}(t) = \begin{cases} \frac{2}{j} & \text{if } j \text{ is odd} \\ 0 & \text{otherwise} \end{cases} \tag{157}$$

from which

$$\int_{-1}^{1} dt f(t) \sim \sum_{j=1}^{p-1} f(t_j)\tilde{w}_j \qquad \tilde{w}_j = \frac{4\sin(j\pi/p)}{p}\sum_{k=0}^{[\![p/2-1]\!]} \frac{\sin((k+1)j\pi/p)}{k+1} \tag{158}$$

14 Ogata quadrature

A very common problem in physics is the study of Fourier transformations in two dimensions. It is often the case that the function that we want to transform is spherically symmetric, or can be made spherically symmetric after some manipulations. Specifically, if we want to study the problem

$$\int d^2\boldsymbol{b} f(\|\boldsymbol{b}\|^2)e^{i\boldsymbol{b}\cdot\boldsymbol{q}} \tag{159}$$

we can use the polar coordinates to reach

$$\int_0^\infty db\ bf(b^2)\int_0^{2\pi} d\theta e^{ibq\cos(\theta)} \equiv 2\pi\int_0^\infty db\ bf(b^2)J_0(bq) \tag{160}$$

where $J_0(x)$ is the Bessel function of the first kind, and it is defined via the given θ integral. Notice that if we had a function g with some angular dependence, we can typically expand such a dependence away into a sum of $\sin, \cos$ multiplied by functions that depend only on b^2. In general, integrals of the form

$$\int_0^{2\pi} d\theta e^{ibq\cos(\theta)}\cos^n(\theta)\sin^m(\theta) \tag{161}$$

can still be written in terms of Bessel J functions, but with different index and possibly some prefactor. Therefore, our goal is to study al algorithm that allows for the evaluation of the integrals

$$\int_0^\infty db\ bf(b^2)J_\nu(bq) \tag{162}$$

This integrals are so fundamental that they take the name of Hankel transform of f! To simplify the notation a bit, we will study the scaled integral

$$\int_0^\infty dx\ g(x)J_\nu(x) \tag{163}$$

where it should be clear that we lost no generality, since the two integrals are connected via simple variable transformations. Let us present first the result, and then derive it.

$$\int_0^\infty dx\ g(x)J_\nu(x) \sim \sum_{n\geq 1} w_n^{(\nu)} g(z_n^\nu) \tag{164}$$

$$z_n^\nu = \frac{\pi}{h}\psi\left(\frac{h\xi_n^{(\nu)}}{\pi}\right) \tag{165}$$

$$w_n^{(\nu)} = \pi\frac{Y_\nu(\xi_n^{(\nu)})}{J_{\nu+1}(\xi_n^{(\nu)})}J_\nu(z_n^\nu)\psi\left(\frac{h\xi_n^{(\nu)}}{\pi}\right) \tag{166}$$

where Y_ν is the Bessel function of the second kind, $\xi_n^{(\nu)}$ are the zeros of the J_ν function, h is a free (!) parameter of the algorithm (but it is typically small) and the ψ function given by

$$\psi(t) = t\tanh\left(\frac{\pi}{2}\sinh(t)\right), \qquad \psi'(t) = \frac{\pi t\cosh(t) + \sinh(\pi\sinh(t))}{1+\cosh(\pi\sinh(t))} \tag{167}$$

which for small t values (i.e. close to zero) starts to grow slowly, and then reached $\psi(t) \sim t$ for t large enough ($t \geq \pi$ we already have that $\psi(t) - t \sim 10^{-16}$).

First, we can start with the underlying interpolant that is used to define most of the quadrature formulas.

We can write it from he first barycentric formula, using the zeros of the Bessel function as nodes. We will study the case of J_0, but the formulas above works for any ν, and we suppress therefore the index ν everywhere. In this case, we know that the monic polynomial is defined by the zeros of the Bessel, by construction, and therefore $l(x) \propto J_0(x)$ up to some irrelevant constant. We also know that the interpolant weights are computed from $J_0'(\xi_n)$. A notable property of the Bessel functions is that

$$J_1(x) = -\frac{d}{dx}J_0(x) \tag{168}$$

therefore, our interpolating polynomial is

$$g(x) = -\sum_{\substack{k=-N\\k\neq 0}}^{N} g(\xi_k)\frac{J_0(x)}{J_1(\xi_k)(x-\xi_k)} \tag{169}$$

where we follow the convention of the index of the zeros of the Bessel function starting from 1 rather than 0. We also note that we are using both the positive zeros and the negative zeros, the former identified by $\xi_{k>0}$, the latter by $\xi_{k<0}$, and that we have $\xi_{-k} = -\xi_k$.

Now, we take a particular integral, that seems unrelated to the problem at hand, but everything will become clear in a moment. We want to take

$$\mathcal{I} = \int_{-\infty}^{\infty} dx\ |x| f(x) = -\sum_{\substack{k=-N\\k\neq 0}}^{N} \frac{f(\xi_k)}{J_1(\xi_k)}\int_{-\infty}^{\infty} dx\ |x|\frac{J_0(x)}{(x-\xi_k)} \tag{170}$$

Now, we need to find the value of the integral

$$\mathcal{I}_k = \int_{-\infty}^{\infty} dx\ |x|\frac{J_0(x)}{(x-\xi_k)} \tag{171}$$

This can be done via a clever use of a specific contour integral in the complex plane for x, showing that it has no poles, and recovering the following

$$\mathcal{I}_k = -\pi|\xi_k|Y_0(\xi_k) \tag{172}$$

that leads to

$$\mathcal{I} = \pi\sum_{\substack{k=-N\\k\neq 0}}^{N} |\xi_k|Y_0(\xi_k)\frac{f(\xi_k)}{J_1(\xi_k)} \tag{173}$$

The integral $\mathcal{I}$ can be rescaled by a simple $x \to hx$ change of variable, that obviously does not change the original expression, but produces

$$\mathcal{I} = h\pi\sum_{\substack{k=-N\\k\neq 0}}^{N} |h\xi_k|Y_0(\xi_k)\frac{f(h\xi_k)}{J_1(\xi_k)} \tag{174}$$

Now, we can go back to the original integral, implementing the variable transformation $x \to \pi\psi(t/\pi)/h$

$$\int_0^{\infty} dx\ g(x)J_\nu(x) = \frac{1}{h}\int_0^{\infty} dt\psi'\left(\frac{t}{\pi}\right)J_0\left(\frac{\pi}{h}\psi\left(\frac{t}{\pi}\right)\right)g\left(\frac{\pi}{h}\psi\left(\frac{t}{\pi}\right)\right) \tag{175}$$

Now, the whole integrand we can interpret as the $|x|f(x)$ that enters in the integral formula $\mathcal{I}$, where we are guaranteed that the extra factor of x is well placed and does not causes problems, since $\psi'(t) \sim \pi t$ for very small t. Therefore, we reach the result

$$\int_0^\infty dx\ g(x)J_\nu(x) = \frac{\pi}{h}\sum_{k\geq 1}^{N}\frac{Y_0(\xi_k)}{J_1(\xi_k)}\psi'\left(\frac{h\xi_k}{h}\right)J_0\left(\frac{\pi}{h}\psi\left(\frac{h\xi_k}{\pi}\right)\right)g\left(\frac{\pi}{h}\psi\left(\frac{h\xi_k}{\pi}\right)\right) \tag{176}$$

where the sum is restricted to positive zeros only because we are studying only the integral over the positive range. In the expression above we recognize the promised result of Eq. (164). The function ψ has been introduced because the argument of the Bessel

$$\frac{\pi}{h}\psi\left(\frac{h\xi_k}{\pi}\right) \tag{177}$$

tends to ξ_k for k large enough, which is a zero of the Bessel, leading to excellent convergence properties of the sum. Note also that the parameter h is used exactly to control how fast the convergence is reached. As some things in life, extremely fast is not always the best choice, and therefore h is typically kept small, say 10^{-3}, so that we have enough terms in the sum to build up a reliable estimation of the result.

This algorithm is not adaptive, since we are forced to use a fix set of nodes. It has however a built-in way to control the accuracy: either one accepts to use tow different values for h, doubling the number of function calls, or one uses the difference of the results with n and $n+m$ nodes as an estimation of the error.

15 On the Cauchy principal Value

There are multiple techniques to handle a principal value in an integral. If one looks at the book [4] together with the implementation in [5], the main proposed strategy is to use a modification of a Clenshaw-Curtis rule. There is however an alternative method, which allow one to re-use any rule that is desired and works for multiple singular weight functions.

The algorithm proposed in the book is essentially confined to study the principal value of functions of the form

$$\mathrm{PV}\int_a^b dx\frac{f(x)}{x-c} \tag{178}$$

and the algorithm takes as input just $f(x)$. This is very (very) limiting. In physics, an extremely common type of principal value integral that needs to be computed is

$$\mathrm{PV}\int_a^b dx\frac{\log^n((x-c)^2)}{x-c}f(x) \tag{179}$$

Note that this type of integral cannot be fed to the routines available to [5] and cannot be integrated directly because of the singularity. To treat them, we can <u>subtract</u> the integral, namely

$$\int_a^b dx\frac{\log^n((x-c)^2)}{x-c}(f(x)-f(c)) + f(c)\mathrm{PV}\int_a^b dx\frac{\log^n((x-c)^2)}{x-c} \tag{180}$$

The latter integral we can take exactly (!) and it gives

$$\mathrm{PV}\int_a^b dx\frac{\log^n((x-c)^2)}{x-c} = \frac{1}{2(n+1)}\left[\log^{n+1}((b-c)^2) - \log^{n+1}((c-a)^2)\right] \tag{181}$$

Therefore we have

$$\begin{aligned}\mathrm{PV}\int_a^b dx\frac{\log^n((x-c)^2)}{x-c}f(x) = &\int_a^b dx\frac{\log^n((x-c)^2)}{x-c}(f(x)-f(c)) \\ &+ \frac{f(c)}{2(n+1)}\left[\log^{n+1}((b-c)^2) - \log^{n+1}((c-a)^2)\right]\end{aligned} \tag{182}$$

And the first integral can be very accurately integrated with normal quadrature rules, with the only precaution of returning zero for the integrand if the quadrature rule asks for exactly $x = c$.

16 On (semi-)infinte intervals of integration

In a case in which one wishes to integrate a function over a (semi-)infinite range, either the function itself comes with some weight for which a rule is known (like Gauss-Hermite quadrature or Ogata) that integrates the function by evaluation on special nodes, or one need to resort to some sort of variable change. Ideally, in these cases, it is known where the function is mostly non-zero, and where the function instead is very close to vanishing.

The idea is to have a simple variable transformation of the form

$$\int_0^\infty dx f(x) = \int_0^1 \frac{dt}{t^2} f((1-t)/t) \tag{183}$$

or

$$\int_{-\infty}^\infty dx f(x) = \int_0^1 dt \left(\frac{1}{t^2} + \frac{1}{(1-t)^2}\right) f(1/t - 1/(1-t)) \tag{184}$$

Note that both transformations should not be taken as the perfect transformations for every case. Specifically, one might need something that follows in a more natural way the shape of the function.

The most trivial case would be a function that is bell shaped (not exactly Gaussian, otherwise one could use Gauss-Hermite directly, but for instance Cauchy distribution), that has the center far away from the origin. For a concrete example, take

$$\int_0^\infty \frac{dx}{\pi} \frac{1}{1+(x-a)^2} = \frac{1}{2} + \frac{\arctan(a)}{\pi} \tag{185}$$

If one uses a GSL ([5]) routine that implements directly the variable transformation above, the integral does produce a result already for $a \sim 10^3$. This is of course a trivial case, because the integral can be solved exactly, but we can use it to illustrate the logic. One variable transformation that can be used is

$$x(t) = a \exp\left[\frac{1}{a} \tan\left(\pi t - \frac{\pi}{2}\right)\right] \tag{186}$$

To exemplify why this is a better choice, Fig.6 shows in red the Cauchy distribution in the variable t via the variable transformation above, and in black the same distribution, but with $x(t) = 1/t - 1$. The plot

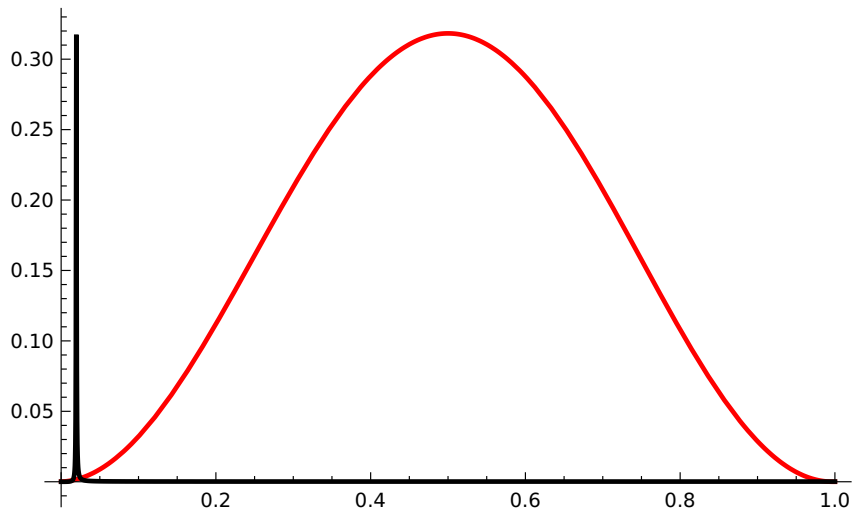

Figure 5: Comparison between variable transformations

has been done for a moderate value of $a = 50$, but the situation is even more extreme for higher values of a. If one accounts also for the jacobian of the transformations, to show the exact function that is going to be integrated, the qualitative conclusion does not change. It should be clear that the red curves are much easier to integrate numerically compared to the black ones.

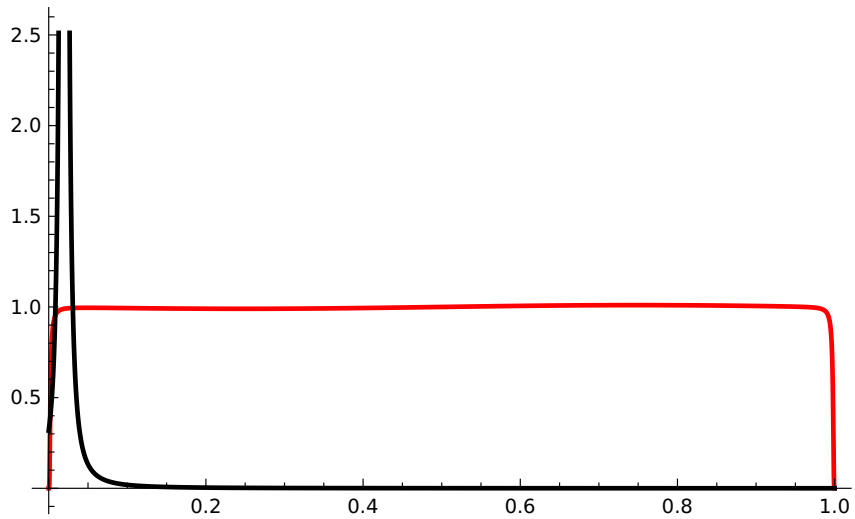

Figure 6: Comparison between variable transformations, with jacobian included

17 Monte Carlo

For higher-dimensional integrals, an application of a normal quadrature rule for each dimension becomes very quickly impossible. This is because we would require an enormous amount of function evaluations. For instance, a 5 dimensional integral, using a non-adaptive 21 points Gauss-Kronrod rule per dimension, would require $21^5 \sim 4 \times 10^6$ function evaluations. Therefore, different methods should be used, and the most common family of methods are Monte-Carlo based methods.

The idea of a Monte Carlo integration is to generate a sample of N random numbers in the integration domain, evaluate the function on said numbers, and give an estimation of the integral as the sample average of the function evaluations. This method also comes with an estimation of the error, by means of the sample standard deviation.

17.1 Crude Monte Carlo

In practice, the most simple approach goes as follow. Assume that we want to integrate a function $f(\boldsymbol{x})$ on a domain Ω:

$$I = \int_\Omega d\boldsymbol{x} f(\boldsymbol{x}) \tag{187}$$

The first step is to make the domain an hyper-rectangle. This can either be achieved by a variable transformation, that should be the preferred method whenever possible, or by simply introducing the characteristic function of the domain

$$\chi_\Omega(\boldsymbol{x}) = \begin{cases} 1 & \text{if } \boldsymbol{x} \in \Omega \\ 0 & \text{otherwise} \end{cases} \tag{188}$$

Then, we can write $g(\boldsymbol{x}) = \chi_\Omega(\boldsymbol{x}) f(\boldsymbol{x})$ and

$$I = V_R \int_R d\boldsymbol{x} \frac{g(\boldsymbol{x})}{V_R} \tag{189}$$

where V_R is the volume of the hyper-rectangle. Now, we can interpret

$$I = V_R \langle g \rangle \tag{190}$$

where the bracket notation signifies the integral mean of the argument. Since we have an integral mean, we know that we can approximate it by

$$I \sim I_N = V_R \mathbb{E}_N \left[g(\boldsymbol{X}) \right] = \frac{V_R}{N} \sum_{j=1}^{N} g(\boldsymbol{x}^{(j)}) \tag{191}$$

where the notation $\mathbb{E}_N$ signifies the sample mean and the capital $\boldsymbol{X}$ signifies that we are now treating $\boldsymbol{X}$ as a random variable. Note that the sample points are generated uniformly in the rectangle.

We can also give an estimation of the variance associated to the result, and use it to give an estimation of the error of the result. We have that

$$\mathrm{Var}[I_N] = \frac{V_R^2}{N^2}\mathrm{Var}\left[\mathbb{E}_N\left[g(\boldsymbol{X})\right]\right] = \frac{V_R^2}{N}\mathrm{Var}[g(\boldsymbol{X})] \tag{192}$$

but the last variance can be computed as the integral variance of the function, namely

$$\mathrm{Var}[g(\boldsymbol{X})] = \langle g^2\rangle - \langle g\rangle^2 = \int_R d\boldsymbol{x}\frac{g^2(\boldsymbol{x})}{V_R} - \left[\int_R d\boldsymbol{x}\frac{g(\boldsymbol{x})}{V_R}\right]^2 \tag{193}$$

which in turn can be approximated by the sample variance

$$\mathrm{Var}[g(\boldsymbol{X})] \sim \sigma_N^2 = \mathbb{V}_N\left[g(\boldsymbol{X})\right] = \frac{1}{N-1}\left[\sum_{j=1}^N g^2(\boldsymbol{x}^{(j)}) - \frac{1}{N}\left(\sum_{j=1}^N g(\boldsymbol{x}^{(j)})\right)^2\right] \tag{194}$$

and

$$\mathrm{Var}[I_N] = \frac{V_R^2}{N(N-1)}\left[\sum_{j=1}^N g^2(\boldsymbol{x}^{(j)}) - \frac{1}{N}\left(\sum_{j=1}^N g(\boldsymbol{x}^{(j)})\right)^2\right] \tag{195}$$

(for the details, see the book [6]).

17.2 Hit-or-miss Monte Carlo

Another possibility is to keep using uniformly generated points, but trying to at least follow the shape of the function. The starting point is still

$$I = \int_R d\boldsymbol{x} g(\boldsymbol{x}) \tag{196}$$

but now we assume that we know the maximum value M that the function $g(\boldsymbol{x})$ can take on the rectangle, i.e. $g(\boldsymbol{x}) \leq M\ \forall \boldsymbol{x} \in R$. Now define the ratio

$$\pi = \frac{I}{V_R} \tag{197}$$

If we can generate an estimation for p, then we can achieve an estimation for the integral. The idea is to generate N tuples of numbers, uniformly distributed in the augmented rectangle $R \times [0, M]$ of the form $(\boldsymbol{x}^{(j)}, y^{(j)})$, and count the number of tuples that have $y^{(j)} \leq g(\boldsymbol{x}^{(j)})$. Call this number N_h. Then

$$\frac{N_h}{N} = p \sim \pi \tag{198}$$

It is clear that the variable p generated is sampled from a binomial distribution with average $N\pi$ and variance $N\pi(1-\pi)$. Therefore, we can construct the estimation of the integral as

$$I_N = pV_R \tag{199}$$

and we can verify that indeed

$$\langle I_N\rangle = V_R\frac{\langle N_h\rangle}{N} = V_R\frac{N\pi}{N} = V_R\pi = I \tag{200}$$

Note that this result means that if we were to repeat the 'measurement' of p with N points a large number of times, then we would find that the average value of those 'measurements' is the true value of the integral. Therefore we conclude that

$$\mathbb{E}_M[I_N] \sim \langle I_N\rangle = I \tag{201}$$

Moreover, with the same idea, we have that

$$\mathrm{Var}[I_N] = \frac{I(V_R - I)}{N} \sim \frac{\mathbb{E}_M[I_N](V_R - \mathbb{E}_M[I_N])}{N} \tag{202}$$

Note that we tacitly assumed that the function to be integrated was non-negative, which can always be achieved by a simple shift of the integrand.

17.3 Importance sampling

One final approach is to modify the crude MC method to generate the sampling points from a distribution that is not uniform, but that follows more closely the structure of the integrand.

Take a function $w(\boldsymbol{x})$ that is roughly $\sim g(\boldsymbol{x})$, and assume that it is positive and normalized on the rectangle of integration. Assume also that we know how to compute the primitive of said function, i.e.

$$W(\boldsymbol{x}) = \int d\boldsymbol{x} w(\boldsymbol{x}), \quad \int_R \boldsymbol{x} w(\boldsymbol{x}) = 1 \tag{203}$$

snd that $w(\boldsymbol{x}) \geq 0 \ \forall \boldsymbol{x} \in R$. Then:

$$I = \int_R d\boldsymbol{x} g(\boldsymbol{x}) = \int_R d\boldsymbol{x} w(\boldsymbol{x}) \ \frac{g(\boldsymbol{x})}{w(\boldsymbol{x})} \tag{204}$$

Now we can interpret $d\boldsymbol{x} w(\boldsymbol{x})$ as a probability measure, and therefore we know that we can estimate the value of the integral using the crude MC method, but now the distribution of sample points is given by $w(\boldsymbol{x})$ instead of the uniform distribution. Therefore we find that

$$I_N = \frac{1}{N} \sum_{j=1}^{N} \frac{g(\boldsymbol{x}^{(j)})}{w(\boldsymbol{x}^{(j)})} \tag{205}$$

$$\mathrm{Var}[I_N] = \frac{1}{N(N-1)} \left[\sum_{j=1}^{N} \frac{g^2(\boldsymbol{x}^{(j)})}{w^2(\boldsymbol{x}^{(j)})} - \frac{1}{N} \left(\sum_{j=1}^{N} \frac{g(\boldsymbol{x}^{(j)})}{w(\boldsymbol{x}^{(j)})} \right)^2 \right] \tag{206}$$

where note that the volume of the hyper-rectangle is no longer present because we are assuming that the $w(\boldsymbol{x})$ distribution is normalized.

Quite trivially, if we choose $w(\boldsymbol{x}) = g(\boldsymbol{x})$ we have vanishing variance, which is equivalent to say that we know how to integrate the function analytically.

Note also that the standard crude MC method is recovered by choosing $w(\boldsymbol{x}) = V_R^{-1}$ which is indeed correctly normalized on the rectangle.

Part III

Integration of Differential Equations

Bibliography

[1] Markus Diehl, Riccardo Nagar, and Frank J. Tackmann. "ChiliPDF: Chebyshev interpolation for parton distributions". In: Eur. Phys. J. C 82.3 (2022), p. 257. DOI: 10.1140/epjc/s10052-022-10223-1. arXiv: 2112.09703 [hep-ph].

[2] Valerio Bertone, Stefano Carrazza, and Juan Rojo. "APFEL: A PDF Evolution Library with QED corrections". In: Comput. Phys. Commun. 185 (2014), pp. 1647–1668. DOI: 10.1016/j.cpc.2014.03.007. arXiv: 1310.1394 [hep-ph].

[3] URL: https://github.com/vbertone/apfelxx/blob/QED/docs/latex/pdf/Interpolation.pdf.

[4] R. Piessens, C.W. Überhuber, and D.K. Kahaner. Quadpack: A Subroutine Package for Automatic Integration. Springer Series in Computational Mathematics. Ediciones Edunsa, 1983. ISBN: 9780387125534. URL: https://books.google.it/books?id=IMhPAQAAIAAJ.

[5] URL: https://www.gnu.org/software/gsl/doc/html/integration.html#qawc-adaptive-integration-for-cauchy-principal-values.

[6] Alberto Rotondi, Paolo Pedroni, and Antonio Pievatolo. Probabilità, Statistica e Simulazione. 3rd ed. UNITEXT. Milano: Springer, 2012. ISBN: 978-88-470-2363-5. DOI: 10.1007/978-88-470-2364-2.